



# SphinxLib

---

*Library functions*

Version 2.1.4 (August 2018)

## Table of Content

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>1. Introduction.....</b>                                      | <b>3</b>  |
| <b>2. SphinxLib functions for Windows and Linux.....</b>         | <b>4</b>  |
| 2.1. General functions.....                                      | 4         |
| 2.2. XML Handling.....                                           | 15        |
| 2.2.1. Introduction.....                                         | 15        |
| 2.2.2. XML-Functions.....                                        | 16        |
| 2.3. Camera functions.....                                       | 25        |
| 2.4. CAnCamGigE-CL functions.....                                | 32        |
| 2.5. Test functions.....                                         | 33        |
| 2.6. Structures.....                                             | 34        |
| <b>3. Appendix.....</b>                                          | <b>38</b> |
| 3.1. Buffer Structure.....                                       | 38        |
| 3.2. Error Codes.....                                            | 40        |
| 3.3. Compiling the library.....                                  | 41        |
| 3.3.1. Windows.....                                              | 41        |
| 3.3.1.1. Library.....                                            | 41        |
| 3.3.1.2. Filterdriver.....                                       | 42        |
| 3.3.2. Linux.....                                                | 42        |
| 3.3.2.1. Directories.....                                        | 42        |
| 3.3.2.2. Install required libraries (as root or using sudo)..... | 42        |
| 3.3.2.3. Build the MathParser linux library.....                 | 43        |
| 3.3.2.4. Build the Sphinx linux library.....                     | 43        |
| 3.3.2.5. Build the linux x-windows demo application.....         | 43        |
| 3.3.2.6. Build the linux console demo application.....           | 43        |
| 3.3.2.7. Build the linux console GenICam demo application.....   | 44        |
| 3.3.2.8. Build the linux QT sphinxGEVviewer application.....     | 44        |
| 3.3.2.9. Build the Sphinx linux filter driver.....               | 44        |
| <b>4. Revision Changes.....</b>                                  | <b>45</b> |
| 4.1. V2.1.3 to V2.1.4.....                                       | 45        |
| 4.2. V2.1.2 to V2.1.3.....                                       | 45        |
| 4.3. V2.1.1 to V2.1.2.....                                       | 45        |
| 4.4. V2.1.0 to V2.1.1.....                                       | 45        |
| 4.5. V2.0.9 to V2.1.0.....                                       | 46        |
| 4.6. V2.0.8 to V2.0.9.....                                       | 47        |
| 4.7. V2.0.7 to V2.0.8.....                                       | 48        |
| 4.8. V2.0.5 to V2.0.6.....                                       | 48        |
| 4.9. V2.0.4 to V2.0.5.....                                       | 48        |
| 4.10. V2.0.2 to V2.0.3.....                                      | 49        |
| 4.11. V1.x.x to V2.x.x.....                                      | 50        |

# 1. Introduction

Sphinx for GigE Vision is an image acquisition library, which provides access to all GigE Vision compliant devices.

It also includes some special functions, related to the Sensor to Image product CANCam-GigE.

Sensor to Image provides a sample implementation in c source code and also a viewer application (SphinxViewer), which could be used as reference for customized applications.

All GigE Vision devices have to provide an xml-file, describing the system features. So the recommended way to write an application for GigE Vision devices is to download and evaluate this xml-file and then activate features, which are supported by the system.

The following document describes the library functions, which are available for Windows or Linux based systems.

You'll find the SphinxLib.dll/sphinxlib.a in the software directory of the provided archive.

## 2. SphinxLib functions for Windows and Linux

This chapter describes the single functions of the C/C++ Library for Windows/Linux. The following example describes and explains the function:

### Sample function

---

|              |                                                                                                                                                                                                                                        |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Function:    | A short description of the function.                                                                                                                                                                                                   |
| Syntax:      | The function's syntax description in C/C++.                                                                                                                                                                                            |
| Description: | This section contains the purpose and the usage of the function. Also the parameters are explained.                                                                                                                                    |
| Return:      | Here you find the type and range of the return values.<br>Many functions return predefined status codes (GEV_STATUS_...).<br>Please check SphinxLib.h for reference.                                                                   |
| Reference:   | List of other routines, which have a relationship to the current function.<br>In addition refer to the GigE Vision specification (GEVS).<br>For detailed error codes of GEV_STATUS_LOCAL_PROBLEM, please use function get_local_error. |

In the directory `Windows/sphinxlib` you will find the SphinxLib library `SphinxLib.dll`, `sphinxlib.lib` and the header-file `SphinxLib.h`

In the directory `Linux/sphinxlib/` you will find the GigE Vision Device library `sphinxlib.a` and the header-file `SphinxLib.h`

The latest version can be obtained from Sensor to Image.

The functions marked as intern, are usually not used in application development. But they are used by other functions and special custom commands.

Following data types are equivalent:

|          |               |                |               |
|----------|---------------|----------------|---------------|
| Windows: | BYTE          | WORD           | DWORD         |
| Linux:   | unsigned char | unsigned short | unsigned long |

Principle of work:

Almost every function corresponds with the feature, referenced by the xml-file. So if a function is used, it uses the registers mentioned in the xml-file.

### Important:

The files in the directory `Windows\SphinxLib\Extras` must be in the same directory as the `SphinxLib.dll`. These files are used by the `SphinxLib.dll`.

The files in the directory `Linux/sphinxlib/extras` must be in the same directory as the application. These files are used by the `sphinxlib.a` library.

DLL Versions: MathParser-1.0.0, libxml2-2.7.8, libxslt-1.1.26

### 2.1. General functions

#### GEVClose

---

|              |                                                                                                                                                                           |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Function:    | Close GigE Vision device                                                                                                                                                  |
| Syntax:      | <code>WORD GEVClose(BYTE cam_nr);</code>                                                                                                                                  |
| Description: | This function closes the communication session (GEV Control Channel) to a GigE Vision device.<br>Parameter <code>cam_nr</code> specifies the GigE Vision device instance. |

---

Return: [see Error Codes](#)

Reference: GEVInit, GEVS

---

### GEVCloseFilterDriver

---

Function: Close GigE Vision device filter driver

Syntax: `WORD GEVCloseFilterDriver(BYTE cam_nr);`

Description: This function closes the S2I filter driver for the stream channel of the specified device  
Parameter *cam\_nr* specifies the GigE Vision device instance.

Return: [see Error Codes](#)

Reference: GEVInitFilterDriver, GEVGetFilterDriverVersion

---

### GEVCloseStreamChannel

---

Function: Close GigE Vision device stream channel

Syntax: `WORD GEVCloseStreamChannel(BYTE cam_nr);`

Description: This function closes the stream channel of the specified device.  
Parameter *cam\_nr* specifies the GigE Vision device instance.

Return: [see Error Codes](#)

Reference: GEVOpenStreamChannel

---

### GEVDiscovery

---

Function: Perform GigE Vision discovery

Syntax: `WORD GEVDiscovery(DISCOVERY *dis,  
DISCOVERY_CALLBACK_FUNC c_func, DWORD d_timeout, BOOL  
ignore_subnet);`

Description: This function can discover up to 20 GigE Vision compliant devices in the network.  
Parameter *dis* returns number and parameters of the found devices ([see 2.6 Structures](#) and SphinxLib.h for the definition of struct DISCOVERY and DEVICE\_PARAM).  
Parameter *d\_timeout* specifies a discovery timeout in ms.  
Parameter *ignore\_subnet* specifies if subnet ignore flag of discovery message should be set.

Return: [see Error Codes](#)

Reference: GEVInit, GEVS

---

### GEVForceIp

---

Function: Force an IP address

Syntax: `WORD GEVForceIp(DWORD new_ip, DWORD subnet, DWORD  
gateway, BYTE *mac);`

Description: This function temporary modifies the network configuration of a device.  
Parameter *new\_ip* sets IP-address, *subnet* sets subnet mask and *gateway* sets the gateway address.  
Parameter *mac* is the MAC address of the device.

Return: [see Error Codes](#)

Reference: GEVSetNetConfig, GEVGetNetConfig, GEVS

---

### GEVGetChannelParameter

---

Function: Returns channel parameter

Syntax: `WORD GEVGetChannelParameter(BYTE cam_nr, CHANNEL_PARAMETER *cparam);`

Description: This function returns the control and stream channel parameter. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *cparam* specifies the channel parameter ([see 2.6 Structures](#) and SphinxLib.h for definition of CHANNEL\_PARAMETER).

Return: [see Error Codes](#)

Reference: GEVSetChannelParameter

---

### GEVGetDetailedLog

---

Function: Gets detailed log

Syntax: `WORD GEVGetDetailedLog(BYTE cam_nr, BYTE *on_off);`

Description: This function returns the detailed log in the parameter on\_off. Parameter *cam\_nr* specifies the GigE Vision device instance.

Possible values of the parameter on\_off

```
#define DETAILED_LOG_OFF          0
Disable all log outputs.

#define DETAILED_LOG_INFO        1
Enable information outputs

#define DETAILED_LOG_WARNING     2
Enable warning outputs

#define DETAILED_LOG_ERROR       4
Enable error outputs

#define DETAILED_LOG_REGISTER    8
Enable register read/write outputs
```

Example:

```
GEVGetDetailedLog(device, &detailed_log);

if((detailed_log & DETAILED_LOG_INFO) == DETAILED_LOG_INFO)
    printf("Info log output is on\n");

if((detailed_log & DETAILED_LOG_ERROR) == DETAILED_LOG_ERROR)
    printf("Error log output is on\n");
```

Return: [see Error Codes](#)

Reference: GEVSetDetailedLog

---

### GEVGetConnectionStatus

---

Function: Returns connection status

Syntax: `WORD GEVGetConnectionStatus(BYTE cam_nr, BYTE *status, BYTE *eval);`

Description: This function returns the connection status. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *status* is a pointer to a variable containing the connection

status. 0 = OK, 1 = timeout, 2 = access denied.  
Parameter *eval* is a pointer to a variable containing the evaluation status. 0 = OK, 1 = evaluation expired.

Return: [see Error Codes](#)

Reference: ---

---

### GEVGetFilterDriverVersion

---

Function: Get S2I filter driver version

Syntax: `WORD GEVGetFilterDriverVersion(BYTE cam_nr, BYTE *version_major, BYTE *version_minor);`

Description: This function returns the S2I filter driver version of the specified device (*cam\_nr*). As the filter driver is bound to each device instance, the parameter *cam\_nr* is needed, although no communication to the device is done.  
Parameter *version\_major* is a pointer to a variable containing the major version.  
Parameter *version\_minor* is a pointer to a variable containing the minor version.  
If the filter driver is not activated the return value in *version\_major* and *version\_minor* is 0.

Return: [see Error Codes](#)

Reference: `GEVInitFilterDriver`, `GEVCloseFilterDriver`

---

### GEVGetHeartbeatRate

---

Function: Returns heartbeat rate

Syntax: `WORD GEVGetHeartbeatRate(BYTE cam_nr, DWORD *heartbeat_rate);`

Description: This function returns heartbeat rate in ms.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *heartbeat\_rate* is a pointer to a variable containing the heartbeat rate.

Return: [see Error Codes](#)

---

### GEVGetMemorySize

---

Function: Get image memory size

Syntax: `WORD GEVGetMemorySize(BYTE cam_nr, DWORD *mem_size);`

Description: This function returns the image acquisition memory-size.  
Parameter *mem\_size* is a pointer to a variable containing the memory size.  
Parameter *cam\_nr* specifies the GigE Vision device instance.

Return: [see Error Codes](#)

Reference: `GEVSetMemorySize`

---

### GEVGetNetConfig

---

Function: Get network configuration

**Syntax:** `WORD GEVGetNetConfig(BYTE cam_nr, BYTE *dhcp, DWORD *ip, DWORD *subnet, DWORD *gateway);`

**Description:** This function reads the network configuration of the device. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *dhcp* returns dhcp enable status, *ip* returns IP-address, *subnet* returns subnet mask and *gateway* returns the gateway address.

**Return:** [see Error Codes](#)

**Reference:** GEVSetNetConfig

---

### GEVGetPixelPtr (Deprecated)

---

**Function:** Returns the logical address of the image memory

**Syntax:** `WORD GEVGetPixelPtr(BYTE cam_nr, DWORD offset, UINT64 *ptr_adr);`

**Description:** This function returns the logical memory address of the local image buffer. This function is deprecated, it is recommended to use *GEVGetImageBuffer*. Parameter *offset* shifts the returned start of the image buffer. It can be used when working with memory divided in several pages for different images. Pointer *ptr\_adr* gives access to the pixeldata. Parameter *cam\_nr* specifies the GigE Vision device instance.  
Example:

```
GEVGetPixelPtr(1,0, &ptradr);
GPixel = (BYTE *)ptradr;
```

**Return:** [see Error Codes](#)

**Reference:** GEVGetImage, GEVGetImageBuffer

---

### GEVGetReadWriteParameter

---

**Function:** Returns read/write parameter

**Syntax:** `WORD GEVGetReadWriteParameter(BYTE cam_nr, DWORD *ack_timeout, BYTE *retry_count);`

**Description:** This function returns read/write parameter for the *GEVReadRegister*, *GEVWriteRegister*, *GEVReadMemory* and *GEVWriteMemory* functions. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *ack\_timeout* is a pointer to a variable containing the acknowledge timeout in ms. Parameter *retry\_count* is a pointer to a variable containing the retry count for the read/write commands.

**Return:** [see Error Codes](#)

**Reference:** GEVSetReadWriteParameter

---

### GEVInit

---

**Function:** Initialize a GigE Vision device



**Syntax:** `WORD GEVInit(BYTE cam_nr, CONNECTION *con, ERROR_CALLBACK_FUNC error_func, BYTE save_xml, BYTE open_mode);`

**Description:** This function opens a control channel to a GigE Vision device. In addition the heartbeat thread is started.  
 With *cam\_nr* an instance number for the camera is assigned.  
 Parameter *con* specifies the connection data ([see 2.6 Structures](#) and SphinxLib.h for definition of CONNECTION).  
 Parameter *error\_func* describes the error callback function. The error function will be executed, when an error occurred.  
 Parameter *save\_xml* specifies the flag to save the xml file to disk. Directory is fixed to the location of the library.  
 Parameter *open\_mode* specifies the control privileges of the device.

```
#define OPEN_ACCESS      0
#define EXCLUSIVE_ACCESS 1
#define CONTROL_ACCESS   2
```

**Return:** [see Error Codes](#)

**Reference:** GEVClose, GEVGetLocalError

---

### GEVInitFilterDriver

**Function:** Init S2I filter driver

**Syntax:** `WORD GEVInitFilterDriver(BYTE cam_nr);`

**Description:** This function activates the S2I stream channel filter driver for the specified device (*cam\_nr*).

**Return:** [see Error Codes](#)

**Reference:** GEVCloseFilterDriver, GEVGetFilterDriverVersion

---

### GEVOpenStreamChannel

**Function:** Open GigE Vision device stream channel

**Syntax:** `WORD GEVOpenStreamChannel(BYTE cam_nr, DWORD ip, WORD port, DWORD multicast);`

**Description:** This function opens the stream channel of the specified GigE Vision device (*cam\_nr*).  
 Parameter *ip* specifies the destination ip address of the stream channel.  
 Parameter *port* specifies the destination port address of the stream channel.  
 Parameter *multicast* specifies the multicast ip address of the stream channel, address 0.0.0.0 disables multicast.

**Return:** [see Error Codes](#)

**Reference:** GEVCloseStreamChannel

---

### GEVReadRegister

**Function:** Receive data from a GigE Vision device

**Syntax:** `WORD GEVReadRegister(BYTE cam_nr, DWORD cmd, BYTE cnt, DWORD *pptr);`

**Description:** This function reads 32bit data from the specified GigE Vision device (*cam\_nr*). This function implements the GEV Read\_Reg command.  
The register address is set in *cmd*.  
The number of 32bit reads is set in *cnt*.  
The parameters itself are accessible through the pointer *pptr*

**Return:** [see Error Codes](#)

**Reference:** GEVSetReadWriteParameter ,GEVWriteRegister, GEVS

---

### GEVReadMemory

---

**Function:** Receive memory data from a GigE Vision device

**Syntax:** `WORD GEVReadMemory(BYTE cam_nr, DWORD maddr, DWORD cnt, DWORD *pptr);`

**Description:** This function reads a memory block from the specified GigE Vision device (*cam\_nr*). This function implements the GEV Read\_Mem command.  
The memory address is given in *maddr*.  
The number of bytes to read is set in *cnt*.  
The memory data itself is accessible through the pointer *pptr*  
To get the write status in percent use the *GEVSetReadWriteMemoryCallback* function to set a callback function.

**Return:** [see Error Codes](#)

**Reference:** GEVWriteMemory, GEVSetReadWriteMemoryCallback, GEVS

---

### GEVSetActionCommand

---

**Function:** Set action command

**Syntax:** `WORD GEVSetActionCommand(BYTE cam_nr, DWORD device_key, DWORD group_key, DWORD group_mask, DWORD action_time);`

**Description:** This function sets the action command.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *device\_key* specifies the device key to authorize the action on this device.  
Parameter *group\_key* specifies the group key to define a group of devices on which the actions have to be executed.  
Parameter *group\_mask* specifies the group mask to be used to filter out some of these devices from the group.

**Return:** [see Error Codes](#)

**Reference:** GEVS

---

### GEVSetChannelParameter

---

**Function:** Set channel parameter

**Syntax:** `WORD GEVSetChannelParameter(BYTE cam_nr, CHANNEL_PARAMETER cparam);`

**Description:** This function sets the control and stream channel parameter.  
Parameter *cam\_nr* specifies the GigE Vision device instance.

Parameter *cparam* specifies the channel parameter ([see 2.6 Structures](#) and `SphinxLib.h` for definition of `CHANNEL_PARAMETER`).

Return: [see Error Codes](#)

Reference: `GEVGetChannelParameter`

---

## GEVSetDetailedLog

Function: Sets detailed log

Syntax: `WORD GEVSetDetailedLog(BYTE cam_nr, BYTE on_off);`

Description: This function sets the detailed log with the parameter *on\_off*. Parameter *cam\_nr* specifies the GigE Vision device instance.

Possible values of the parameter *on\_off*

```
#define DETAILED_LOG_OFF          0
Disable all log outputs.

#define DETAILED_LOG_INFO         1
Enable information outputs

#define DETAILED_LOG_WARNING      2
Enable warning outputs

#define DETAILED_LOG_ERROR        4
Enable error outputs

#define DETAILED_LOG_REGISTER     8
Enable register read/write outputs
```

Example: Enable info and error log outputs

```
GEVSetDetailedLog(device, DETAILED_LOG_INFO | DETAILED_LOG_ERROR);
```

Return: [see Error Codes](#)

Reference: `GEVGetDetailedLog`

---

## GEVSetHeartbeatRate

Function: Set heartbeat rate

Syntax: `WORD GEVSetHeartbeatRate(BYTE cam_nr, DWORD heartbeat_rate);`

Description: This function sets the heartbeat timeout register and the timeout value for connection checking.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *heartbeat\_rate* is the heartbeat timeout value in ms. The heartbeat timeout register of the device is set to this value. The interval value to check the connection state is set to half of the heartbeat timeout value. With the function *GEVGetChannelParameter* you can read the interval value for the connection check. (`CHANNEL_PARAMETER cc_heartbeat_timeout`.)

Return: [see Error Codes](#)

---

## GEVSetMemorySize (Deprecated)

Function: Set image memory size

Syntax: `WORD GEVSetMemorySize(BYTE cam_nr, DWORD mem_size);`

**Description:** This function reallocates memory as framebuffer. The reference to the pixel data must be updated with `get_pixel_ptr`!  
Parameter *mem\_size* is the memory size in bytes.  
Parameter *cam\_nr* specifies the GigE Vision device instance.

This function is deprecated and is no longer needed if image buffer is handled in user application and provided to the library via *GEVGetImageBuffer*.

**Return:** [see Error Codes](#)

**Reference:** `GEVGetMemorySize`, `GEVGetPixelPtr`, `GEVGetImageBuffer`

---

### GEVSetMessageChannel

---

**Function:** Bind function to message channel event

**Syntax:** `WORD GEVSetMessageChannel(BYTE cam_nr, unsigned short port, MESSAGECHANNEL_CALLBACK_FUNC c_func);`

**Description:** This function binds a custom function to the message channel, which will be executed, when an event occurred.

Parameter *cam\_nr* specifies the GigE Vision device instance.

Parameter *port* specifies the message channel port to open.

Parameter *c\_func* describes the callback function.

`BYTE (WINAPI *MESSAGECHANNEL_CALLBACK_FUNC) (BYTE cam_nr, int event_id, int data_length, BYTE *data);`

**Return:** [see Error Codes](#)

**Reference:** -

---

### GEVSetNetConfig

---

**Function:** Set network configuration

**Syntax:** `WORD GEVSetNetConfig(BYTE cam_nr, BYTE dhcp, DWORD ip, DWORD subnet, DWORD gateway);`

**Description:** This function modifies the network configuration of a GigE Vision device.

Parameter *cam\_nr* specifies the GigE Vision device instance.

Parameter *dhcp* sets the dhcp enable status, *ip* sets IP-address, *subnet* sets the subnet mask and *gateway* sets the gateway address.

**Return:** [see Error Codes](#)

**Reference:** `GEVGetNetConfig`

---

### GEVSetReadWriteMemoryCallback

---

**Function:** Associate function with read/write memory functions

**Syntax:** `WORD GEVSetReadWriteMemoryCallback(BYTE cam_nr, READ_WRITE_MEM_CALLBACK_FUNC c_func);`

**Description:** This function associates a custom function to get the read/write status in percent.

Parameter *cam\_nr* specifies the GigE Vision device instance.

Parameter *c\_func* describes the callback function.

`BYTE (WINAPI *READ_WRITE_MEM_CALLBACK_FUNC) (BYTE cam_nr, int s_cnt);`

**Return:** [see Error Codes](#)

**Reference:** `GEVReadMemory`, `GEVWriteMemory`

---

## GEVSetReadWriteParameter

---

**Function:** Sets read/write parameter

**Syntax:** `WORD GEVSetReadWriteParameter(BYTE cam_nr, DWORD ack_timeout, BYTE retry_count);`

**Description:** This function sets the read/write parameter for the parameter for the *GEVReadRegister*, *GEVWriteRegister*, *GEVReadMemory* and *GEVWriteMemory* functions. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *ack\_timeout* is the acknowledge timeout in ms. Parameter *retry\_count* is the number of retries if read/write command wasn't acknowledged.

**Return:** [see Error Codes](#)

**Reference:** `GEVGetReadWriteParameter`

---

## GEVTestPacket

---

**Function:** Trigger GigE Vision test packet

**Syntax:** `WORD GEVTestPacket(BYTE cam_nr, DWORD *packet_size);`

**Description:** This function triggers the specified GigE Vision device (*cam\_nr*) to send a test packet. This feature can be used to find maximum packet size in a network environment. Parameter *packet\_size* returns the packet size of the test packet. Size of the test packet is equal to the stream channel packet size setting.

**Return:** [see Error Codes](#)

**Reference:** `GEVTestFindMaxPacketSize`

---

## GEVSetTraversingFirewallsInterval

---

**Function:** Set the interval in seconds for traversing firewalls

**Syntax:** `WORD GEVSetTraversingFirewallsInterval(BYTE cam_nr, DWORD interval);`

**Description:** This function sets the interval in seconds for traversing firewalls to the specified GigE Vision device (*cam\_nr*). The parameter *interval* is the time in seconds between the packets to be sent to the stream channel. Traversing firewalls is only available when the stream channel source port register is available in the device.

**Return:** [see Error Codes](#)

**Reference:** `GEVOpenStreamChannel`

---

## GEVWriteMemory

---

**Function:** Send memory data to a GigE Vision Device device

**Syntax:** `WORD GEVWriteMemory(BYTE cam_nr, DWORD maddr, DWORD cnt, DWORD *pptr);`

**Description:** This function writes a memory block to the specified GigE Vision device (*cam\_nr*). This function implements the GEV Write\_Mem command. The memory address is given in *maddr*. The number of bytes to write is set in *cnt*.

The data itself is accessible through the pointer *pptr*  
To get the write status in percent use the  
*GEVSetReadWriteMemoryCallback* function to set a callback function.

Return: [see Error Codes](#)

Reference: *GEVReadMemory*, *GEVSetReadWriteMemoryCallback*

---

## GEVWriteRegister

---

Function: Send register data to GigE Vision Device device

Syntax: `WORD GEVWriteRegister(BYTE cam_nr, DWORD cmd, BYTE  
cnt, DWORD *pptr);`

Description: This function writes 32bit data to the specified GigE Vision device  
(*cam\_nr*). This function implements the GEV Write\_Reg command.

The register address is set in *cmd*.

The number of 32bit words to be written is set in *cnt*.

The parameters itself are accessible through the pointer *pptr*.

Example:

```
error = GEVWriteRegister(CAM_0,  
                        0xA100,  
                        1,  
                        &reg_data);
```

Return: [see Error Codes](#)

Reference: *GEVReadRegister*

## 2.2. XML Handling

### 2.2.1. Introduction

Part of the GigE Vision standard is that each device has to provide a xml-file, which describes the functions of the device. This xml-file contains a set of nodes, where each node has a type and a unique name. Nodes can be linked to others and each connection plays a certain role. The syntax and elements of this xml-file is described in the GenICam GenAPI standard document available at [www.genicam.org](http://www.genicam.org).

The S2I-software, which evaluates the xml file has some restrictions, so not all features of the xml are supported right now.

#### Supported interface and node types:

- Integer (IntReg, MaskIntReg, Integer)
- String (StringReg)
- Command
- Boolean
- Category
- Port
- Enumeration
- StructReg
- SwissKnife
- IntSwissKnife
- Float (FloatReg)
- EnumEntry
- pVariable
- Formula
- FormulaTo
- FormulaFrom
- Converter
- IntConverter
- Register

#### Supported node elements:

- Address
- pAddress
- Value
- pValue
- pVariable
- Min
- pMin
- Max
- pMax
- CommandValue
- AccessMode
- OnValue
- OffValue
- LSB
- MSB
- Bit

- Inc
- pInc
- Representation
- Endianness
- Length
- Visibility
- pIsImplemented
- pIsAvailable
- DisplayName
- ToolTip
- Unit
- pIsLocked
- Sign
- pIndex
- pValueIndexed
- ValueIndexed
- ValueDefault
- pValueDefault
- pValueCopy
- Slope
- pOffset
- pLength
- DisplayNotation
- DisplayPrecision
- pSelected

That means only nodes and elements listed above are accessible through following functions.

From application side it is also possible to use the GenICam reference implementation for xml handling. Please refer to sample application for.

### 2.2.2. XML-Functions

#### GEVGetFeatureBoolean

---

Function: Returns the Boolean value of a dedicated feature, described in xml file

Syntax: `WORD GEVGetFeatureBoolean(BYTE cam_nr, char *feature_name, DWORD *bool_value);`

Description: This functions returns the Boolean value of a feature described in xml-file.

Parameter *cam\_nr* specifies the GigE Vision device instance.

Parameter *feature\_name* contains the feature name (e.g. LUTEnable)

Parameter *bool\_value* returns the Boolean value.

Return: [see Error Codes](#)

Reference: GEVSetFeatureBoolean, GEVGetFeatureParameter

#### GEVGetFeatureCommand

---

Function: Returns the command value of a dedicated feature, described in xml file

Syntax: `WORD GEVGetFeatureCommand(BYTE cam_nr, char *feature_name, DWORD *cmd_value);`



**Description:** This function returns the command value of a feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature node name (e.g. AcquisitionStart).  
Parameter *cmd\_value* returns the command value.

**Return:** [see Error Codes](#)

**Reference:** GEVSetFeatureCommand, GEVGetFeatureParameter

---

### GEVGetFeatureDisplayName

---

**Function:** Returns the display name of a dedicated feature, described in xml file

**Syntax:** `WORD GEVGetFeatureDisplayName(BYTE cam_nr, char *feature_name, char *display_name, int display_name_length);`

**Description:** This function returns the display name of a feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature name (e.g. Width)  
Parameter *display\_name* returns the display name.  
Parameter *display\_name\_length* contains length of the display name pointer.

**Return:** [see Error Codes](#)

**Reference:** GEVGetFeatureTooltip

---

### GEVGetFeatureEnableStatus

---

**Function:** Returns the enable status of a dedicated feature, described in xml file

**Syntax:** `WORD GEVGetFeatureEnableStatus(BYTE cam_nr, char *feature_name, BYTE *enable);`

**Description:** This function returns the access status of a feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature name (e.g. Width)  
Parameter *enable* returns the status.

**Return:** [see Error Codes](#)

**Reference:** --

---

### GEVGetFeatureEnumeration

---

**Function:** Returns the enumeration name of a dedicated feature, described in xml file

**Syntax:** `WORD GEVGetFeatureEnumeration(BYTE cam_nr, char *feature_name, char *enum_name, int str_len);`

**Description:** This function returns the current enumeration value of an enumeration feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature name (e.g. Pixelformat)  
Parameter *enum\_name* returns feature enumeration value.  
Parameter *str\_len* contains length of feature name pointer.

**Return:** [see Error Codes](#)

Reference: GEVSetFeatureEnumeration , GEVGetFeatureEnumerationName, GEVGetFeatureParameter

### GEVGetFeatureEnumerationName

---

Function: Returns the name of an enumeration value

Syntax: `WORD GEVGetFeatureEnumerationName(BYTE cam_nr, char *feature_name, BYTE enum_index, char *enum_name, int str_len);`

Description: This function returns an enumeration value indexed by `enum_index` of a feature described in xml-file.  
 Parameter `cam_nr` specifies the GigE Vision device instance.  
 Parameter `feature_name` contains feature name (e.g. PixelFormat)  
 Parameter `enum_index` contains enumeration index.  
 Parameter `enum_name` returns feature name of enumeration name. (e.g. Mono8)  
 If returned string length of `enum_name` is equal 0, then enumeration name is not available.  
 Parameter `str_len` contains length of feature name pointer.  
 For a complete list of possible enum values call `GEVGetFeatureParameter`.

Return: [see Error Codes](#)

Reference: GEVSetFeatureEnumeration, GEVGetFeatureEnumeration, GEVGetFeatureParameter

### GEVGetFeatureFloat

---

Function: Returns the float value of a dedicated feature, described in xml file

Syntax: `WORD GEVGetFeatureFloat(BYTE cam_nr, char *feature_name, float *float_value);`

Description: This function returns the float value of a feature described in xml-file.  
 Parameter `cam_nr` specifies the GigE Vision device instance.  
 Parameter `feature_name` contains the feature name (e.g. DeviceTemperature)  
 Parameter `float_value` returns the float value.

Return: [see Error Codes](#)

Reference: GEVSetFeatureFloat

### GEVGetFeatureInteger

---

Function: Returns the integer value of a dedicated feature, described in xml file

Syntax: `WORD GEVGetFeatureInteger(BYTE cam_nr, char *feature_name, DWORD *int_value);`

Description: This function returns the integer value of a feature described in xml-file.  
 Parameter `cam_nr` specifies the GigE Vision device instance.  
 Parameter `feature_name` contains the feature name (e.g. Width)  
 Parameter `int_value` returns feature integer value.

Return: [see Error Codes](#)

Reference: GEVSetFeatureInteger, GEVGetFeatureParameter

---

## GEVGetFeatureInvalidator

---

- Function:** Returns the invalidator of a dedicated feature, described in xml file
- Syntax:** `WORD GEVGetFeatureInvalidator(BYTE cam_nr, char *feature_name, BYTE index, char *invalidator_name, int str_len);`
- Description:** This function returns the invalidator of a feature described in xml-file. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *feature\_name* contains feature name (e.g. Width) Parameter *index* contains the invalidator index. To obtain the total number of invalidators, use the function *GEVGetFeatureParameter*. Parameter *invalidator\_name* returns invalidator name. Parameter *str\_len* contains length of invalidator\_name pointer.
- Return:** [see Error Codes](#)
- Reference:** *GEVGetFeatureParameter*

---

## GEVGetFeatureList

---

- Function:** Returns the pointer of a dedicated feature list, described in xml file
- Syntax:** `WORD GEVGetFeatureList(BYTE cam_nr, FeatureListPtr *featureListPtr, BYTE *maxLevel);`
- Description:** This function returns a pointer to the feature list described in xml-file. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *featureListPtr* contains the feature list. XMLs have a hierarchical structure. Parameter *maxLevel* returns the maximal level to list.
- ```
error = GEVGetFeatureList(camera, &featureListPtr,
&maxLevel);
if(featureListPtr == NULL)
    return;

while(featureListPtr != NULL)
{
    for(i = 0; i < featureListPtr->Level; i++)
        printf("\t");
    printf("%s\n", featureListPtr->Name);
    featureListPtr = featureListPtr->Next;
}
```
- Return:** [see Error Codes](#)
- Reference:** ---

---

## GEVGetFeatureParameter

---

- Function:** Returns properties of a dedicated feature, described in xml file
- Syntax:** `WORD GEVGetFeatureParameter(BYTE cam_nr, char *feature_name, FEATURE_PARAMETER *f_param);`
- Description:** This function returns properties of a device feature described in xml-file. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *feature\_name* contains feature name (e.g. Width) Parameter *f\_param* returns the parameters of the given feature ([see 2.6 Structures](#) and *Sphinxlib.h* for definition of struct *FEATURE\_PARAMETER*).

Example:

```
error = GEVGetFeatureParameter(camera,
                               feature_str,
                               &f_param);
printf(„Feature-Value-Min: %d\n“, f_param.Min);
printf(„Feature-Value-Max: %d\n“, f_param.Max);
...
```

Return: [see Error Codes](#)

Reference: ---

## GEVGetFeaturePort

Function: Returns the port of a dedicated feature, described in xml file

Syntax: `WORD GEVGetFeaturePort(BYTE cam_nr, char *feature_name, char *port_name, int port_name_length);`

Description: This function returns the port of a feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature name (e.g. Width)  
Parameter *port\_name* returns port name.  
Parameter *port\_name\_length* contains length of *port\_name* pointer.

Return: [see Error Codes](#)

Reference: ---

## GEVGetFeatureRegister

Function: Returns the values of a dedicated register feature, described in xml file

Syntax: `WORD GEVGetFeatureRegister(BYTE cam_nr, char *feature_name, DWORD len, BYTE *pbuffer);`

Description: This function returns values of a register feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature name (e.g. LUTValueAll)  
Parameter *len* contains length of buffer/register.  
Parameter *pbuffer* returns buffer values.

Return: [see Error Codes](#)

Reference: `GEVSetFeatureRegister`, `GEVGetFeatureParameter`

## GEVGetFeatureString

Function: Returns the string value of a dedicated feature, described in xml file

Syntax: `WORD GEVGetFeatureString(BYTE cam_nr, char *feature_name, char *str_value);`

Description: This function returns the string value of a feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature name (e.g. DeviceVersion)  
Parameter *str\_value* returns feature string value.

Return: [see Error Codes](#)

Reference: `GEVSetFeatureString`, `GEVGetFeatureParameter`

---

## GEVGetFeatureTooltip

---

**Function:** Returns the tooltip string of a dedicated feature, described in xml file

**Syntax:** `WORD GEVGetFeatureTooltip(BYTE cam_nr, char *feature_name, char *tooltip_name, int tooltip_name_length);`

**Description:** This function returns the tooltip string of a feature described in xml-file. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *feature\_name* contains feature name (e.g. Width). Parameter *tooltip\_name* returns tooltip string. Parameter *tooltip\_name\_length* contains length of tooltip\_name pointer.

**Return:** [see Error Codes](#)

**Reference:** GEVGetFeatureDisplayName

---

## GEVGetFeatureUnit

---

**Function:** Returns unit string of a dedicated feature, described in xml file

**Syntax:** `WORD GEVGetFeatureUnit(BYTE cam_nr, char *feature_name, char *unit_name, int unit_name_length);`

**Description:** This function returns a unit string of a feature described in xml-file. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *feature\_name* contains feature name (e.g. DeviceTemperature). Parameter *unit\_name* returns unit string (e.g. C). Parameter *unit\_name\_length* contains length of unit\_name pointer.

**Return:** [see Error Codes](#)

**Reference:** ---

---

## GEVGetXmlFile

---

**Function:** Get the xml file

**Syntax:** `WORD GEVGetXmlFile(BYTE cam_nr, BYTE **xmlfile);`

**Description:** This function returns the xml-file of the device. Before using this function, call GEVGetXmlSize to get the size of the xml file and allocate memory (*xmlfile*). Parameter *cam\_nr* specifies the GigE Vision device instance.

**Return:** [see Error Codes](#)

**Reference:** GEVGetXmlSize

---

## GEVGetXmlSize

---

**Function:** Get the size of the device xml file

**Syntax:** `WORD GEVGetXmlSize(BYTE cam_nr, DWORD *size);`

**Description:** This function returns the size of the xml-file of the device. The xml file size is stored in the parameter *size*. Call this function before using GEVGetXmlFile. Parameter *cam\_nr* specifies the GigE Vision device instance.

**Return:** [see Error Codes](#)

Reference: GEVGetXmlFile

---

### GEVInitXml

---

Function: Initialize xml

Syntax: `WORD GEVInitXml(BYTE cam_nr);`

Description: This function gets the xml-file from the device and builds the feature list in the library.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
This function also needs the schema files in the library/program path.  
The schema files can you find in the SphinxLib\Extra directory.

Return: [see Error Codes](#)

Reference: GEVSetXmlFile, GEVSetSchemaPath

---

### GEVSetFeatureBoolean

---

Function: Set the Boolean value of a dedicated feature, described in xml file

Syntax: `WORD GEVSetFeatureBoolean(BYTE cam_nr, char *feature_name, DWORD bool_value);`

Description: This function writes the Boolean value of a feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature name (e.g. LUTEnable)  
Parameter *bool\_value* contains new Boolean value.

Return: [see Error Codes](#)

Reference: GEVGetFeatureBoolean, GEVGetFeatureParameter

---

### GEVSetFeatureCommand

---

Function: Set command value of a dedicated feature, described in xml file

Syntax: `WORD GEVSetFeatureCommand(BYTE cam_nr, char *feature_name, DWORD cmd_value);`

Description: This function writes the command value of a feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature name (e.g. AcquisitionStart)  
Parameter *cmd\_value* contains command value.

Return: [see Error Codes](#)

Reference: GEVGetFeatureCommand, GEVGetFeatureParameter

---

### GEVSetFeatureEnumeration

---

Function: Set enumeration name of a dedicated feature, described in xml file

Syntax: `WORD GEVSetFeatureEnumeration(BYTE cam_nr, char *feature_name, char *enum_name, int str_len);`

Description: This function writes an enumeration value to a feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature name (e.g. Pixelformat)  
Parameter *enum\_name* contains the feature's enumeration name (e.g.

Mono8).

Parameter *str\_len* contains length of feature name pointer.

Return: [see Error Codes](#)

Reference: GEVGetFeatureEnumeration, GEVGetFeatureEnumerationName, GEVGetFeatureParameter

---

### GEVSetFeatureFloat

Function: Set float value of a dedicated feature, described in xml file

Syntax: `WORD GEVSetFeatureFloat(BYTE cam_nr, char *feature_name, float float_value);`

Description: This function writes a float value to a feature described in xml-file. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *feature\_name* contains feature name (e.g. ExposureTime) Parameter *float\_value* contains new float value.

Return: [see Error Codes](#)

Reference: GEVGetFeatureFloat

---

### GEVSetFeatureInteger

Function: Set integer value of a dedicated feature, described in xml file

Syntax: `WORD GEVSetFeatureInteger(BYTE cam_nr, char *feature_name, DWORD int_value);`

Description: This function writes an integer value of a feature described in xml-file. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *feature\_name* contains feature name (e.g. Width) Parameter *int\_value* contains the new integer value.

Return: [see Error Codes](#)

Reference: GEVGetFeatureInteger, GEVGetFeatureParameter

---

### GEVSetFeatureRegister

Function: Set register values of a dedicated feature, described in xml file

Syntax: `WORD GEVSetFeatureRegister(BYTE cam_nr, char *feature_name, DWORD len, BYTE *pbuffer);`

Description: This function writes buffer values to a register feature described in xml-file. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *feature\_name* contains feature name (e.g. LUTValueAll) Parameter *len* contains the length of buffer. Parameter *pbuffer* contains the new buffer values.

Return: [see Error Codes](#)

Reference: GEVGetFeatureRegister, GEVGetFeatureParameter

---

### GEVSetFeatureString

Function: Set string value of a dedicated feature, described in xml file

Syntax: `WORD GEVSetFeatureString(BYTE cam_nr, char *feature_name, char *str_value);`

Description: This function writes the string value of a feature described in xml-file.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *feature\_name* contains feature name (e.g. DeviceUserID)  
Parameter *str\_value* contains the new string value.

Return: [see Error Codes](#)

Reference: GEVGetFeatureInteger, GEVGetFeatureParameter

---

### GEVSetSchemaPath

---

Function: Set the path of the schema files

Syntax: `WORD GEVSetSchemaPath(BYTE cam_nr, char *schema_path);`

Description: This function sets the new path of the schema files.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *schema\_path* contains the new path of the schema files.  
Default path of the schema files are current program/library path.  
This function must be called after GEVInit function and before the GEVInitXml function.

Return: [see Error Codes](#)

Reference: GEVInitXml

---

### GEVSetXmlFile

---

Function: Set xml file in the library

Syntax: `WORD GEVSetXmlFile(BYTE cam_nr, char *xml_name);`

Description: This function sets the xml-file in the library.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *xml\_name* contains path to new xml file to read and use in the library.

Return: [see Error Codes](#)

Reference: GEVInitXml



## 2.3. Camera functions

### GEVAcquisitionStart

**Function:** Start image acquisition

**Syntax:** `WORD GEVAcquisitionStart(BYTE cam_nr , DWORD number_images_to_acquire);`

**Description:** This function starts the image acquisition and writes network image data to a PC ringbuffer. Transfer of images to image memory has to be done by *GEVGetImage* (deprecated) or *GEVGetImageBuffer*. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *number\_images\_to\_acquire* contains the number of images to grab. This parameter is used only, if AcquisitionMode is set to MultiFrame and node AcquisitionFrameCount is not available. After that number, GEVAcquisitionStop is called automatically.

| AcquisitionMode | number_images_to_acquire                                  |
|-----------------|-----------------------------------------------------------|
| Continuous      | 0 (unlimited)                                             |
| SingleFrame     | 1                                                         |
| MultiFrame      | 1..n<br>Get the value from the AcquisitionFrameCount node |

If GEV\_STATUS\_FEATURE\_NOT\_AVAILABLE error is returned, please check if one of these entries is not in the XML file.

- AcquisitionMode (if Multi is set then also Acquisition FrameCount should be present)
- Width
- Height
- PixelFormat
- PayloadSize
- AcquisitionStart

**Return:** [see Error Codes](#)

**Reference:** GEVAcquisitionStop, GEVGetImage

### GEVAcquisitionStartEx

**Function:** Start image acquisition without library internal xml handling

**Syntax:** `WORD GEVAcquisitionStartEx(BYTE cam_nr , DWORD number_images_to_acquire, DWORD image_size);`

**Description:** This function initializes an image ring buffer and its parameters. It starts the image acquisition thread. Transfer of images to image memory has to be done by *GEVGetImage* (deprecated) or *GEVGetImageBuffer*. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *number\_images\_to\_acquire* contains the number of images to grab. This parameter is used only, if AcquisitionMode is set to MultiFrame and node AcquisitionFrameCount is not available. After that

number GEVAcquisitionStop is called automatically.  
 Parameter *image\_size* contains the size of one payload block (image).  
 Use this function if xml handling is done outside of sphinxlib. Do not forget to send acquisition start command to the device.

| AcquisitionMode | number_images_to_acquire |
|-----------------|--------------------------|
| Continuous      | 0 (unlimited)            |
| SingleFrame     | 1                        |
| MultiFrame      | 1..n                     |

Return: [see Error Codes](#)

Reference: GEVAcquisitionStart , GEVAcquisitionStop, GEVGetImage

---

### GEVAcquisitionStop

Function: Stop image acquisition

Syntax: `WORD GEVAcquisitionStop(BYTE cam_nr);`

Description: This function stops the image acquisition tread and sends acquisition stop command to the device, if xml handling is done by sphinxlib. Parameter *cam\_nr* specifies the GigE Vision device instance.

Return: [see Error Codes](#)

Reference: GEVAcquisitionStart, GEVGetImage

---

### GEVGetBufferCount

Function: Returns number of buffers of library managed ring buffer

Syntax: `WORD GEVGetBufferCount(BYTE cam_nr, WORD *count);`

Description: Parameter *cam\_nr* specifies the GigE Vision device instance. The parameter *count* returns the number of buffers of the image ring buffer.

Return: [see Error Codes](#)

Reference: GEVSetBufferCount

---

### GEVGetImage (Deprecated)

Function: Get an image from internal ring buffer and copy it to internal lib memory

Syntax: `WORD GEVGetImage(BYTE cam_nr, IMAGE_HEADER *image_header)`

Description: Get an image form the ring buffer and copy it to the internal image memory.  
 The pointer to the internal image memory you can get it with the function *GEVGetPixelPtr*.  
 Parameter *cam\_nr* specifies the GigE Vision device instance.  
 Parameter *image\_header* returns information of the image ([see 2.6 Structures](#) and SphinxLib.h for definition of IMAGE\_HEADER).  
 This function is deprecated and is no longer needed if image buffer is

handled in user application and provided to the library via *GEVGetImageBuffer*.

Return: [see Error Codes](#)

Reference: GEVAcquisitionStart, GEVAcquisitionStop, GEVGetPixelPtr

---

### GEVGetImageBuffer

---

Function: Get an image from the library managed ring buffer and copy it to user memory

Syntax: `WORD GEVGetImageBuffer(BYTE cam_nr, IMAGE_HEADER *image_header, BYTE *image_buffer)`

Description: Get an image from the ring buffer and transmit it to the *image\_buffer* memory pointer. This has to be allocated by the application. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *image\_header* returns information of the image ([see 2.6 Structures](#) and SphinxLib.h for definition of IMAGE\_HEADER).

.Return: [see Error Codes](#)

Reference: GEVAcquisitionStart, GEVAcquisitionStop,

---

### GEVGetImageRingBuffer

---

Function: Get an image from user managed ring buffer and return buffer index

Syntax: `WORD GEVGetImageRingBuffer(BYTE cam_nr, IMAGE_HEADER *image_header, BYTE *index)`

Description: Get an image from the user ring buffer and return the *index* of buffer part. The ring buffer has to be allocated by the application. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *image\_header* returns information of the image ([see 2.6 Structures](#) and SphinxLib.h for definition of IMAGE\_HEADER).

Note: Not work with filter driver, is not implemented in the filter driver.

Return: [see Error Codes](#)

Reference: GEVAcquisitionStart, GEVAcquisitionStop, GEVSetRingBuffer, GEVQueueRingBuffer, GEVReleaseRingBuffer

---

### GEVGetImageFPS

---

Function: Returns the number of the acquired frames per second

Syntax: `WORD GEVGetImageFPS(BYTE cam_nr, double *fps);`

Description: This function returns the number of acquired frames per second. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *fps* returns the frames per second.

Return: [see Error Codes](#)

Reference: ---

---

### GEVGetPacketResend

---

Function: Gets packet resend activation status

Syntax: `WORD GEVGetPacketResend(BYTE cam_nr, BYTE *enable);`

Description: Parameter *cam\_nr* specifies the GigE Vision device instance.  
The parameter *enable* returns the packet resend activation status:  
0 = disabled, 1 = enabled.

Return: [see Error Codes](#)

Reference: GEVSetPacketResend

---

### GEVGetPacketsOutOfOrder

---

Function: Returns the number of handled packets out of order

Syntax: `WORD WINAPI GEVGetPacketsOutOfOrder(BYTE cam_nr, BYTE *packets_out_of_order);`

Description: Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *packets\_out\_of\_order* returns the number of packets taken into account, before sending packet resend requests.

Return: [see Error Codes](#)

Reference: GEVSetPacketsOutOfOrder

---

### GEVGetSecureTransfer

---

Function: Returns secure transfer mode enable status

Syntax: `WORD GEVGetSecureTransfer(BYTE cam_nr, BYTE *enable);`

Description: In Secure Transfer Mode packet resend is sent until the current block is complete. Next block shall be sent by the device not before receiving an acknowledge command.  
This function returns the enable status of secure transfer mode.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *enable* is a pointer to a variable containing the enable status (0 = disable, 1 = enable).

Return: [see Error Codes](#)

Reference: GEVSetSecureTransfer

---

### GEVPacketResend

---

Function: Send packet resend command

Syntax: `WORD GEVPacketResend(BYTE cam_nr, WORD stream_channel, WORD block_id, DWORD first_packet_id, DWORD last_packet_id)`

Description: This function sends the packet resend command.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *stream\_channel* contains the number of the stream channel.  
(0 is first stream channel)  
Parameter *block\_id* contains the block id.  
Parameter *first\_packet\_id* contains the first packet to resend.  
Parameter *last\_packet\_id* contains the last packet to resend.

Example: packet 100 -105 of block 20 shall be resent:

```
GEVPacketResend(CAM_0, 0, 20, 100, 105);
```

Return: [see Error Codes](#)

Reference: GEVSetPacketResend, GEVGetPacketResend

---

## GEVQueueRingBuffer

---

**Function:** Queue an user managed ring buffer part for acquisition of new data

**Syntax:** `WORD GEVQueueRingBuffer(BYTE cam_nr, BYTE image_buffer_index);`

**Description:** If data acquisition ring buffer is managed by user, a buffer part is blocked for user processing by *GEVGetImageRingBuffer*. This function releases this user ring buffer part with index *image\_buffer\_index* for acquisition of new data. Parameter *cam\_nr* specifies the GigE Vision device instance.

Note: Not work with filter driver, is not implemented in the filter driver.

**Return:** [see Error Codes](#)

**Reference:** GEVSetRingBuffer, GEVReleaseRingBuffer, GEVImageRingBuffer

---

## GEVReleaseRingBuffer

---

**Function:** Decouple user managed ring buffer from the library

**Syntax:** `WORD GEVReleaseRingBuffer(BYTE cam_nr);`

**Description:** This function decouples the user ring buffer from the library. Parameter *cam\_nr* specifies the GigE Vision device instance.

Note: Not work with filter driver, is not implemented in the filter driver.

**Return:** [see Error Codes](#)

**Reference:** GEVSetRingBuffer, GEVQueueRingBuffer, GEVImageRingBuffer

---

## GEVSetBufferCount

---

**Function:** Sets number of buffers in library managed ring buffer

**Syntax:** `WORD GEVSetBufferCount(BYTE cam_nr, WORD count);`

**Description:** This function sets the number of buffers (*count*) in the library managed ring buffer. Default setting is 4 buffers. Parameter *cam\_nr* specifies the GigE Vision device instance.

**Return:** [see Error Codes](#)

**Reference:** GEVGetBufferCount

---

## GEVSetPacketResend

---

**Function:** Enables or disables packet resend feature

**Syntax:** `WORD GEVSetPacketResend(BYTE cam_nr, BYTE enable);`

**Description:** This function enables or disables the packet resend feature. Parameter *cam\_nr* specifies the GigE Vision device instance. The parameter *enable* sets the packet resend status: 0 = disable, 1 = enable.

**Return:** [see Error Codes](#)

**Reference:** GEVGetPacketResend

---

## GEVSetPacketsOutOfOrder

---

- Function:** Sets the number of handled out of order packets
- Syntax:** `WORD WINAPI SEVGetPacketsOutOfOrder(BYTE cam_nr, BYTE packets_out_of_order);`
- Description:** Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *packets\_out\_of\_order* sets the number of packets taken into account, before sending packet resend requests. This function is only effective if packet resend is enabled.
- Return:** [see Error Codes](#)
- Reference:** GEVGetPacketsOutOfOrder

---

## GEVSetRingBuffer

---

- Function:** Couple user managed ring buffer with the library
- Syntax:** `WORD GEVSetRingBuffer(BYTE cam_nr, WORD index, void *buffer);`
- Description:** This function sets user ring buffer in the library. Parameter *index* contains the index for the user ring buffer. Parameter *buffer* contains the user ring buffer to set in the library. Parameter *cam\_nr* specifies the GigE Vision device instance.  
  
Note: Not work with filter driver, is not implemented in the filter driver.
- Return:** [see Error Codes](#)
- Reference:** GEVReleaseRingBuffer, GEVQueueRingBuffer, GEVImageRingBuffer

---

## GEVSetSecureTransfer

---

- Function:** Enable secure transfer mode
- Syntax:** `WORD GEVSetSecureTransfer(BYTE cam_nr, BYTE enable, SECURE_TRANSFER_CALLBACK_FUNC c_func);`
- Description:** In Secure Transfer Mode packet resend is sent until the current block is complete. Next block shall be sent by the device not before receiving an acknowledge command.  
This function sets the secure transfer mode. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *enable* enables or disables (0 = disable, 1 = enable) the secure transfer mode. Parameter *c\_func* describes the callback function for secure transfer acknowledge.

### Example:

```
error = GEVGetFeatureEnumeration(pparams->device,
"TransferOperationMode",s1 , sizeof(s1));

if(error == GEV_STATUS_SUCCESS)
{
    if(strcmp(s1,"Secure") == 0)
    {
        // set secure transfer in the SphinxLib
        GEVSetSecureTransfer(pparams->device, 1,
secure_transfer_callback_func);

        secure_transfer = 1;
    }
}
```

```
        else
            secure_transfer = 0;
    }

BYTE WINAPI secure_transfer_callback_func(BYTE cam_nr)
{
    WORD error;
    FEATURE_PARAMETER f_param;

    error = GEVGetFeatureParameter(cam_nr, "TransferAcknowledge",
    &f_param);

    if(error)
    {
        ...
        return(1);
    }
    error = GEVSetFeatureCommand(cam_nr, "TransferAcknowledge",
    f_param.CommandValue);

    if(error)
    {
        ...
        return(1);
    }
    return(0);
}
```

Return: [see Error Codes](#)

Reference: GEVGetSecureTransfer

## 2.4. CANCamGigE-CL functions

Following functions are available only for CANCamGigE-CL CameraLink to GigEVision converters.

### CANCamGetClinkStatus

---

**Function:** Returns status of CameraLink serial interface

**Syntax:** `WORD CANCamGetClinkStatus(BYTE cam_nr, CLINK_STATUS *status);`

**Description:** This function returns the *status* of the CameraLink's serial interface. Parameter *cam\_nr* specifies the GigE Vision device instance. See Sphinxlib.h for the definition of CLINK\_STATUS.

**Return:** [see Error Codes](#)

**Reference:** CANCamSendClink, CANCamReceiveClink

### CANCamInitClinkSerial

---

**Function:** Initializes CameraLink serial interface

**Syntax:** `WORD CANCamInitClinkSerial(BYTE cam_nr, DWORD baud);`

**Description:** This function sets baudrate of the CameraLink serial interface. Parameter *cam\_nr* specifies the GigE Vision device instance. Valid values for *baud* rate are: 9600, 19200, 38400, 57600, 115200.

**Return:** [see Error Codes](#)

**Reference:** CANCamSendClink, CANCamReceiveClink

### CANCamReceiveClink

---

**Function:** Receive data over CameraLink serial interface

**Syntax:** `WORD CANCamReceiveClink(BYTE cam_nr, WORD count_buffer, BYTE *recv_buffer, WORD *recv_count);`

**Description:** This function reads receive buffer of CameraLink's serial interface. Parameter *cam\_nr* specifies the GigE Vision device instance. Parameter *count\_buffer* specifies maximum number of characters to be read. Parameter *recv\_buffer* points to the received characters. Parameter *recv\_count* returns the number of characters read.

**Return:** [see Error Codes](#)

**Reference:** CANCamSendClink

### CANCamSendClink

---

**Function:** Send data over CameraLink serial interface

**Syntax:** `WORD CANCamSendClink(BYTE cam_nr, WORD count, BYTE *send_buffer);`

**Description:** This function writes *count* characters, given by *send\_buffer* to serial sender. Parameter *cam\_nr* specifies the GigE Vision device instance.

**Return:** [see Error Codes](#)

**Reference:** CANCamReceiveClink



## 2.5. Test functions

### GEVTestPacketResend

---

- Function:** Tests the packet resend function
- Syntax:** `WORD GEVTestPacketResend(BYTE cam_nr, BYTE on_off, WORD packet_number, WORD count);`
- Description:** This function tests the packet resend by requesting one or more selectable packets.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *on\_off* switches the packet resend test on or off.  
Parameter *packet\_number* specifies the start of packets to resend.  
Parameter *count* specifies the number of packets to resend. Packet Resend must be enabled with function *GEVSetPacketResend*. Also enable the DETAILED \_LOG\_INFO flag in the *GEVSetDetailedLog* function to see the results.
- Return:** [see Error Codes](#)
- Reference:** ---

### GEVTestFindMaxPacketSize

---

- Function:** Find the maximum of packet size
- Syntax:** `WORD GEVTestFindMaxPacketSize(BYTE cam_nr, WORD *packet_size, WORD ps_min, WORD ps_max, WORD ps_inc);`
- Description:** This function finds the maximum of packet size.  
Parameter *cam\_nr* specifies the GigE Vision device instance.  
Parameter *packet\_size* returns the maximum found packet size.  
Parameter *ps\_min* (minimum packetsize), *ps\_max* (maximum packetsize) and *ps\_inc* (increment) shall be read from the xml and passed to this function.
- Return:** [see Error Codes](#)
- Reference:** GEVTestPacket

## 2.6. Structures

### DISCOVERY

---

Count: Number of devices found.  
 Param: Struct of found device parameter (DEVICE\_PARAM)

### DEVICE\_PARAM

---

IP: ip address of the device  
 manuf: manufacturer name of the device  
 mode: model name of the device  
 version: version of the device  
 AdapterIP: ip address of the network adapter  
 AdapterMask: mask of the network adapter  
 Mac: mac address of the device  
 subnet: subnet mask of the device  
 gateway: gateway of the device adapter\_name: network adapter name  
 serial: serial number of the device  
 userdef\_name: user defined name  
 status: connection status of the device

```
#define DISCOVERY_STATUS_OK 0
#define DISCOVERY_STATUS_ALLREADY_OPEN 1
#define DISCOVERY_STATUS_NOT_SAME_SUBNET 2
#define DISCOVERY_STATUS_CONTROL_OPEN 3
```

### CONNECTION

---

IP\_CANCam: ip address of the device  
 PortData: socket port of the stream channel (value of 0 set port automatic)  
 PortCtrl; socket port of the control channel (value of 0 set port automatic)  
 AdapterIP: ip address of the network adapter  
 AdapterMask: subnet mask of the network adapter  
 adapter\_name: name of the network adapter

### CHANNEL\_PARAMETER

---

cc\_heartbeat\_timeout: control channel heartbeat timeout counter in milliseconds  
 cc\_timeout: control channel timeout in milliseconds  
 cc\_retry: control channel retry count  
 sc\_timeout: stream channel timeout in milliseconds  
 sc\_packet\_resend: stream channel packet resend count  
 sc\_image\_wait\_timeout: stream channel wait of an image timeout in milliseconds

## FeatureList

---

Next: pointer to next feature

Index: index of feature (1,2,3,...)

Name: name of the feature

Type: type of the feature

```
#define TYPE_CATEGORY 0
#define TYPE_FEATURE 1
#define TYPE_INTEGER 2
#define TYPE_FLOAT 3
#define TYPE_STRING 4
#define TYPE_ENUMERATION 5
#define TYPE_COMMAND 6
#define TYPE_BOOLEAN 7
#define TYPE_REGISTER 8
#define TYPE_PORT 9
```

Level: level of the feature

feature 1 -> level 0

    feature 1.1 -> level 1

    feature 1.2 -> level 1

        feature 1.2.1 -> level 2

        feature 1.2.2 -> level 2

feature 2 -> level 0

    feature 2.1 -> level 1

    feature 2.2 -> level 1

        feature 2.2.1 -> level 2

        feature 2.2.2 -> level 2

## FEATURE\_PARAMETER

---

Type: type of the feature

```
#define TYPE_CATEGORY 0
#define TYPE_FEATURE 1
#define TYPE_INTEGER 2
#define TYPE_FLOAT 3
#define TYPE_STRING 4
#define TYPE_ENUMERATION 5
#define TYPE_COMMAND 6
#define TYPE_BOOLEAN 7
#define TYPE_REGISTER 8
#define TYPE_PORT 9
```

Min: minimum value of a integer feature

Max: maximum value of a integer feature

OnValue: OnValue of a Boolean node (see GenICam Standard)

OffValue: OffValue of a Boolean node (see GenICam Standard)

AccessMode: access mode of the feature

```
#define ACCESS_MODE_RO    0x524F // read only
#define ACCESS_MODE_RW    0x5257 // read/write
#define ACCESS_MODE_WO    0x574F // write only
```

**Representation:** The Representation element gives a hint about how to display the integer.

```
// Slider with linear behaviour
#define REPRESENTATION_LINEAR    0
// Slider with logarithmic behaviour
#define REPRESENTATION_LOGARITHMIC    1
// Checkbox
#define REPRESENTATION_BOOLEAN    2
// Decimal number in an edit control
#define REPRESENTATION_PURE_NUMBER    3
// Hex number in an edit control
#define REPRESENTATION_HEX_NUMBER    4
// Undefined Representation
#define REPRESENTATION_UNDEFINDED    5
```

**Inc:** increment of an integer feature

**CommandValue:** command value of a command node

**Length:** length of the feature in bytes

**EnumerationCount:** count of enumeration node

**Visibility:** The Visibility element defines the user level that should get access to the feature

```
#define VISIBILITY_INVISIBLE    0
#define VISIBILITY_BEGINNER    1
#define VISIBILITY_EXPERT    2
#define VISIBILITY_GURU    3
```

**FloatMin:** minimum value of a float feature

**FloatMax:** maximum value of a float feature

**IsImplemented:**

**IsAvailable:**

**IsLocked:** The IsImplemented, IsAvaliable, and IsLocked elements contains the names of nodes implementing an Integer interface.

**Sign:** The Sign element can have the value *Singed* or *Unsigned*.

```
#define SIGN_UNSIGNED    0
#define SIGN_SIGNED    1
```

**Address:** Address of the feature

**DisplayNotation:** Notation of the display

```
#define DISPLAY_NOTATION_AUTOMATIC    0
#define DISPLAY_NOTATION_FIXED    1
#define DISPLAY_NOTATION_SCIENTIFIC    2
```

**DisplayPrecision:** Precision of the display

**InvalidatorCount:** Number of invalidators of the feature

---

PollingTime:                      Polling time of the feature

## **IMAGE\_HEADER**

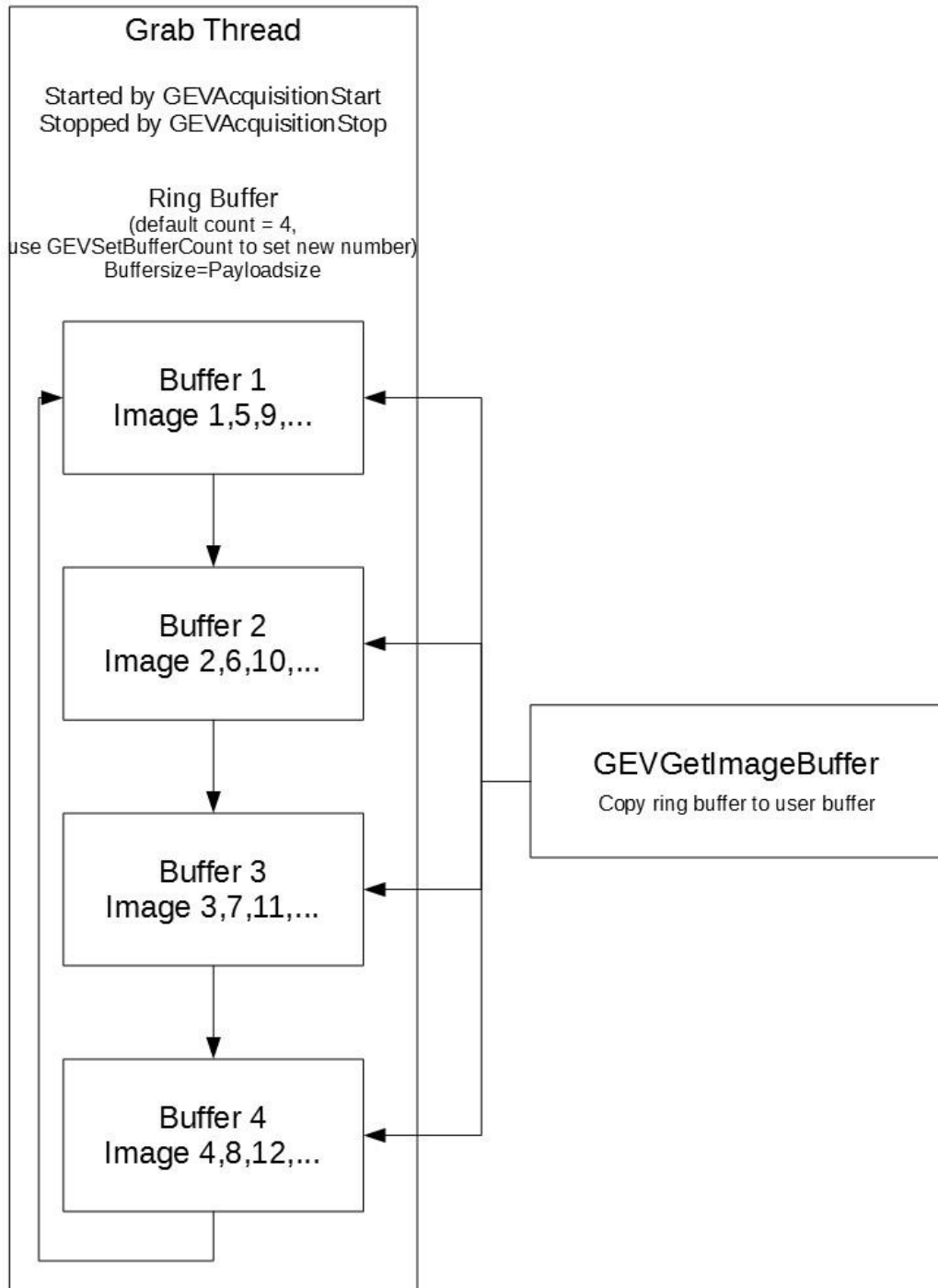
---

|                        |                                                 |
|------------------------|-------------------------------------------------|
| FrameCounter:          | Current frame counter                           |
| TimeStamp:             | Timestamp of the image                          |
| PixelType              | Pixel type of the image                         |
| SizeX:                 | Width in pixels of the image                    |
| SizeY:                 | Height in lines of the image                    |
| OffsetX:               | Offset in pixels from the image origin          |
| OffsetY:               | Offset in lines from the image origin           |
| PaddingX:              | Horizontal padding expressed in bytes           |
| PaddingY:              | Vertical padding expressed in bytes             |
| MissingPacket:         | Number of missing packets                       |
| PayloadType            | Payload Type                                    |
| ChunkDataPayloadLength | Length of all the chunks data payload in bytes. |
| ChunkLayoutId          | Chunk layout ID                                 |

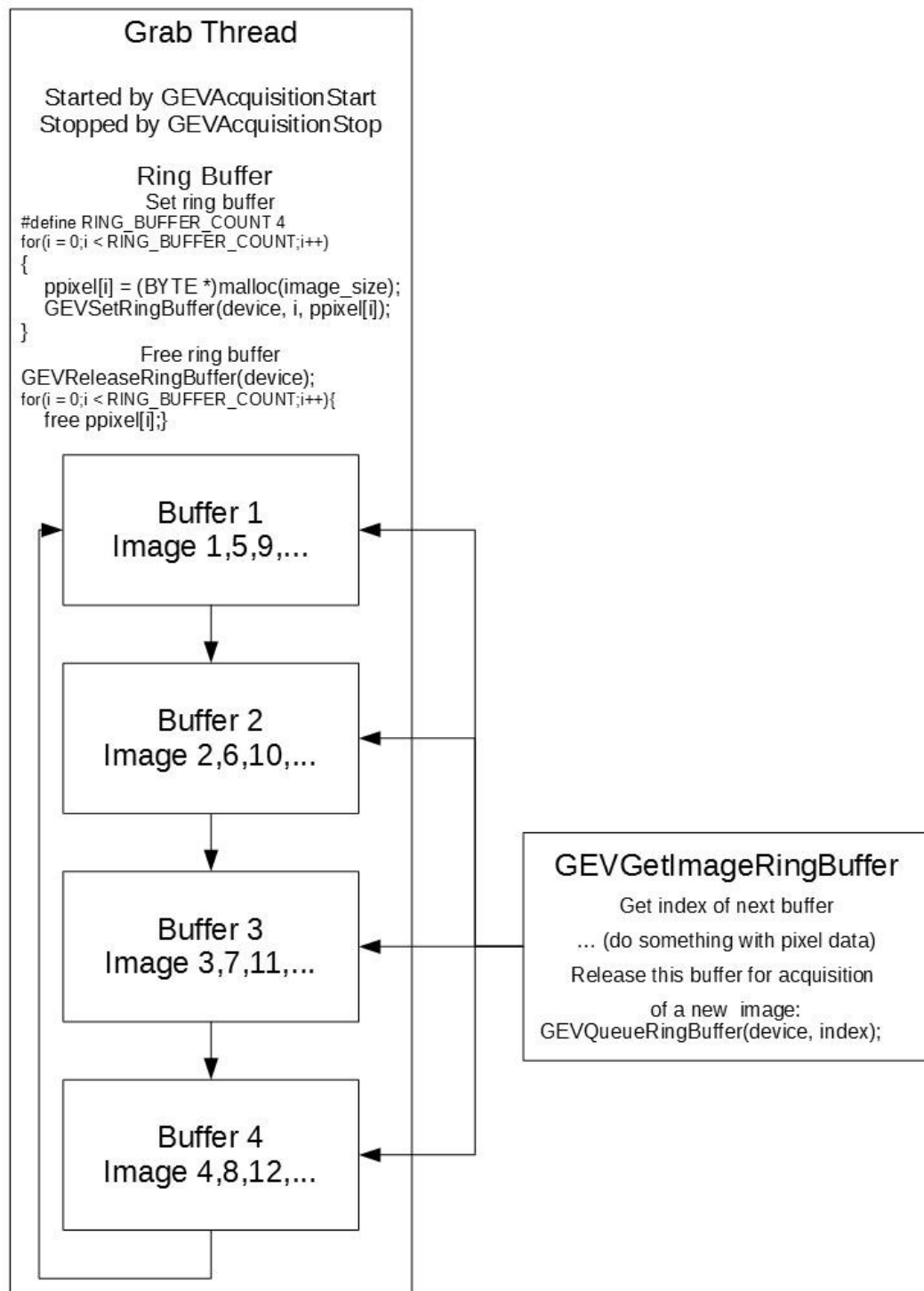
## 3. Appendix

### 3.1. Buffer Structure

Ring buffer managed by SphinxLib



## Ring buffer managed by user



## 3.2. Error Codes

| Status Value                            | Value  | Description                                                                                                |
|-----------------------------------------|--------|------------------------------------------------------------------------------------------------------------|
| GEV_STATUS_SUCCESS                      | 0x0000 | Operation executed successfully                                                                            |
| GigE Vision Status Codes                | 0x8001 |                                                                                                            |
|                                         | ...    |                                                                                                            |
|                                         | 0x8FFF | See GigE Vision Specification                                                                              |
| GEV_STATUS_CAMERA_NOT_INIT              | 0xC001 | Device was not opened with the function GEVInit                                                            |
| GEV_STATUS_CAMERA_ALWAYS_INIT           | 0xC002 | Device was not closed with the function GEVClose                                                           |
| GEV_STATUS_CANNOT_CREATE_SOCKET         | 0xC003 | Can't create Socket port                                                                                   |
| GEV_STATUS_SEND_ERROR                   | 0xC004 | Error sending a GigE Vision command                                                                        |
| GEV_STATUS_RECEIVE_ERROR                | 0xC005 | Error receiving a GigE Vision acknowledge                                                                  |
| GEV_STATUS_CAMERA_NOT_FOUND             | 0xC006 | Device not found (no longer used)                                                                          |
| GEV_STATUS_CANNOT_ALLOC_MEMORY          | 0xC007 | Error to allocate memory                                                                                   |
| GEV_STATUS_TIMEOUT                      | 0xC008 | Timeout to get data from the socket port                                                                   |
| GEV_STATUS_SOCKET_ERROR                 | 0xC009 | Socket port error                                                                                          |
| GEV_STATUS_INVALID_ACK                  | 0xC00A | Command acknowledge invalid                                                                                |
| GEV_STATUS_CANNOT_START_THREAD          | 0xC00B | Thread could not be started                                                                                |
| GEV_STATUS_CANNOT_SET_SOCKET_OPT        | 0xC00C | Failed to set a socket option                                                                              |
| GEV_STATUS_CANNOT_OPEN_DRIVER           | 0xC00D | Driver could not be opened. Check if driver is installed or you don't have administrator privileges        |
| GEV_STATUS_HEARTBEAT_READ_ERROR         | 0xC00E | Heartbeat read error (no longer used)                                                                      |
| GEV_STATUS_EVALUATION_EXPIRED           | 0xC00F | Evaluation expired                                                                                         |
| GEV_STATUS_GRAB_ERROR                   | 0xC010 | Image could not be completely transferred. <a href="#">See IMAGE_HEADER struct</a> MissingPacket parameter |
| GEV_STATUS_DRIVER_READ_ERROR            | 0xC011 | Error communicate with the driver                                                                          |
| GEV_STATUS_XML_READ_ERROR               | 0xC012 | Error read xml file                                                                                        |
| GEV_STATUS_XML_OPEN_ERROR               | 0xC013 | Error open xml file                                                                                        |
| GEV_STATUS_XML_FEATURE_ERROR            | 0xC014 | Feature error (no longer used)                                                                             |
| GEV_STATUS_XML_COMMAND_ERROR            | 0xC015 | Feature command error (no longer used)                                                                     |
| GEV_STATUS_GAIN_NOT_SUPPORTED           | 0xC016 | Gain feature not supported (no longer used)                                                                |
| GEV_STATUS_EXPOSURE_NOT_SUPPORTED       | 0xC017 | Exposure feature not supported (no longer used)                                                            |
| GEV_STATUS_CANNOT_GET_ADAPTER_INFO      | 0xC018 | Can't get information from network adapter                                                                 |
| GEV_STATUS_ERROR_INVALID_HANDLE         | 0xC019 | Invalid handle                                                                                             |
| GEV_STATUS_CLINK_SET_BAUD               | 0xC01A | Error to set Clink baud rate                                                                               |
| GEV_STATUS_CLINK_SEND_BUFFER_FULL       | 0xC01B | Clink send buffer is full                                                                                  |
| GEV_STATUS_CLINK_RECEIVE_BUFFER_NO_DATA | 0xC01C | No data in receive buffer available                                                                        |
| GEV_STATUS_FEATURE_NOT_AVAILABLE        | 0xC01D | Feature not available                                                                                      |
| GEV_STATUS_MATH_PARSER_ERROR            | 0xC01E | Math parser error                                                                                          |
| GEV_STATUS_FEATURE_ITEM_NOT_AVAILABLE   | 0xC01F | Feature item not available (enum entry).                                                                   |
| GEV_STATUS_NOT_SUPPORTED                | 0xC020 | Not supported.                                                                                             |
| GEV_STATUS_GET_URL_ERROR                | 0xC021 | Error reading the url string of the device                                                                 |
| GEV_STATUS_READ_XML_MEM_ERROR           | 0xC022 | Error read xml file of the device                                                                          |
| GEV_STATUS_XML_SIZE_ERROR               | 0xC023 | Size of xml file is wrong.                                                                                 |
| GEV_STATUS_XML_ZIP_ERROR                | 0xC024 | Error unzip xml file                                                                                       |
| GEV_STATUS_XML_ROOT_ERROR               | 0xC025 | Unable to get xml root element of the xml file                                                             |
| GEV_STATUS_XML_FILE_ERROR               | 0xC026 | Xml file is corrupt.                                                                                       |
| GEV_STATUS_DIFFERENT_IMAGE_HEADER       | 0xC027 | Stream image header is wrong.                                                                              |
| GEV_STATUS_XML_SCHEMA_ERROR             | 0xC028 | Error while parsing the XML with the schema file                                                           |
| GEV_STATUS_XML_STYLESHEET_ERROR         | 0xC029 | Error while parsing the XML with the stylesheet file                                                       |
| GEV_STATUS_FEATURE_LIST_ERROR           | 0xC02A | Failed to create the feature list                                                                          |
| GEV_STATUS_ALREADY_OPEN                 | 0xC02B | Device is already open.                                                                                    |
| GEV_STATUS_TEST_PACKET_DATA_ERROR       | 0xC02C | Data from test packet is corrupt                                                                           |
| GEV_STATUS_FEATURE_NOT_FLOAT            | 0xC02D | The feature is not a float feature                                                                         |



|                                |        |                                                                                                                                   |
|--------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------|
| GEV_STATUS_FEATURE_NOT_INTEGER | 0xC02E | The feature is not a integer feature                                                                                              |
| GEV_STATUS_XML_DLL_NOT_FOUND   | 0xC02F | Xml library libxml2.dll not found. Copy libxml2.dll to same directory as SphinxLib or set dll path with SetDllDirectory function. |
| GEV_STATUS_XML_NOT_INIT        | 0xC030 | XML was not initialized with GEVInitXml                                                                                           |
| GEV_STATUS_NOT_SAME_SUBNET     | 0xC031 | device is not in the same subnet as network card                                                                                  |

### 3.3. Compiling the library

#### 3.3.1. Windows

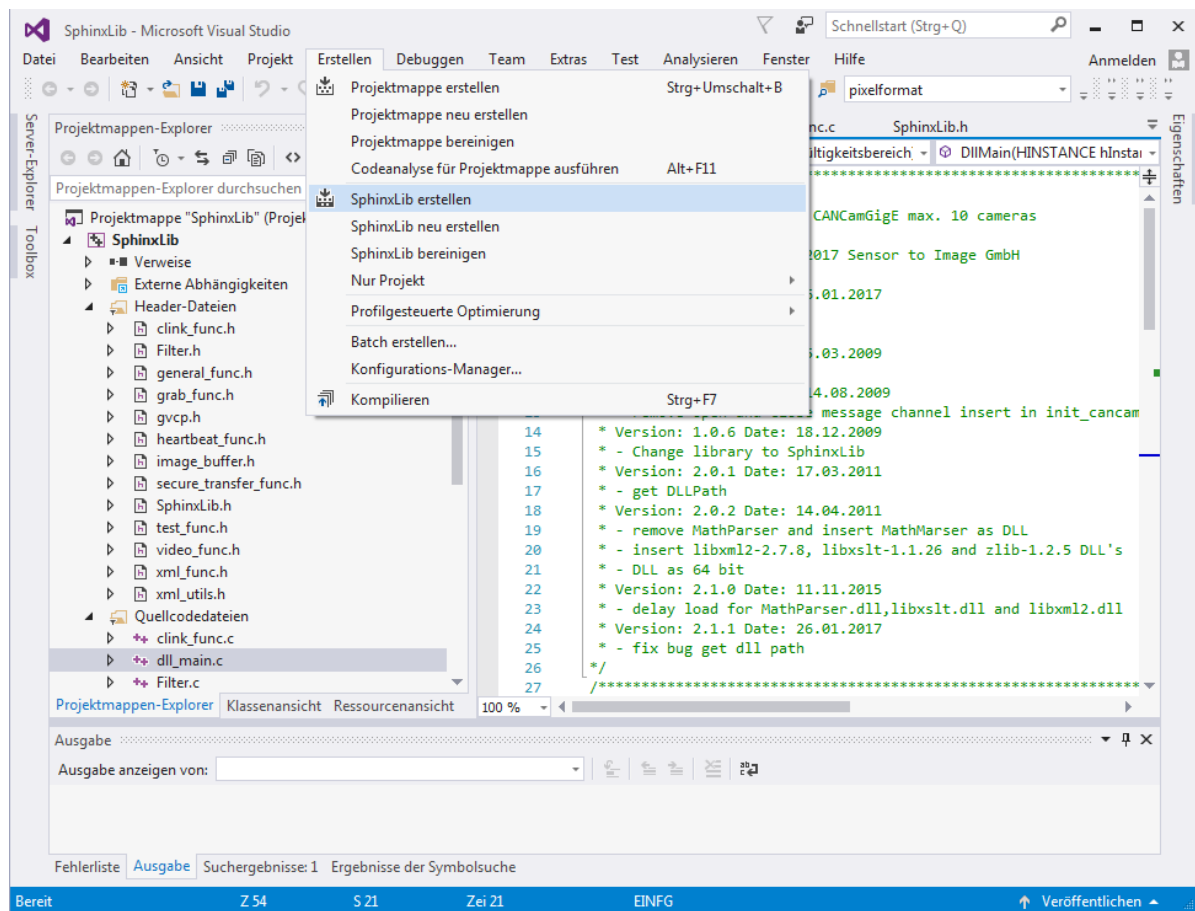
##### 3.3.1.1. Library

The library is provided as a complete project for Visual Studio 2015. Other compilers may be able to import that project or a new project has to be created.

There is available a free version of Visual Studio, called Visual Studio Express.  
(see <https://www.visualstudio.com/de/vs/visual-studio-express/>

or

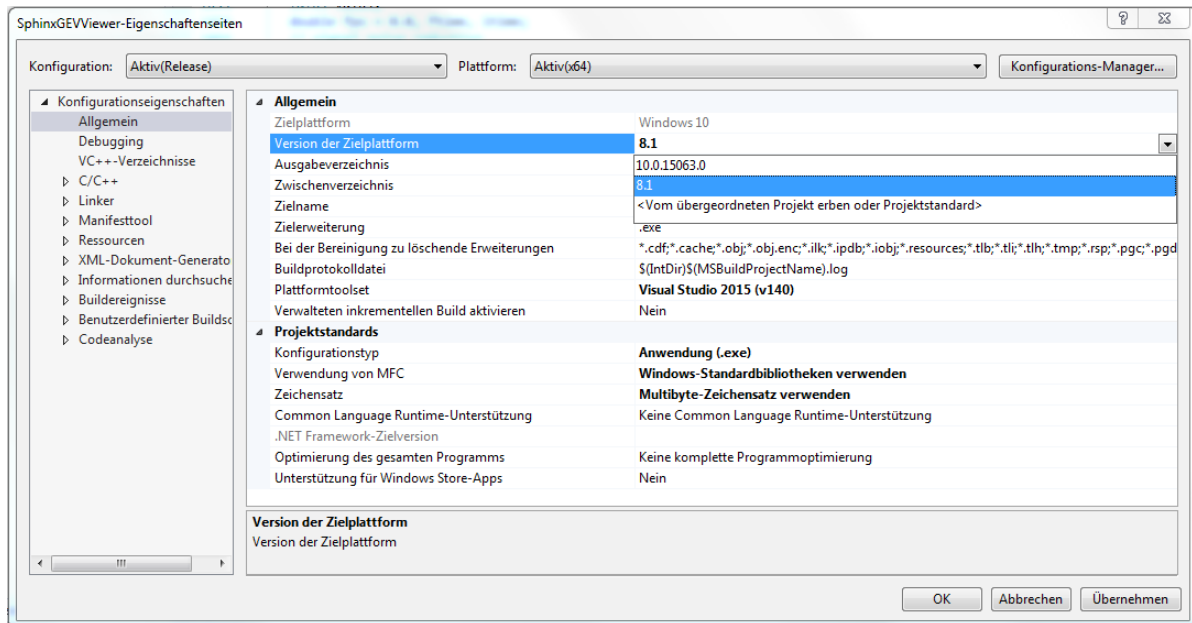
<https://developer.microsoft.com/en-us/windows/hardware/windows-driver-kit>)



Build Library in Visual Studio 2015

#### Note:

If you have the windows sdk 8.1 installed you have to change in the settings to 8.1.



### 3.3.1.2. Filterdriver

Compilation of filter driver needs Microsoft Windows Driver Kit (WDK 10)  
(see <https://developer.microsoft.com/en-us/windows/hardware/windows-driver-kit>)

Use the visual studio project in the FilterDriverSourceCode6 directory to compile filter driver.

## 3.3.2. Linux

### 3.3.2.1. Directories

|                       |   |                                        |
|-----------------------|---|----------------------------------------|
| console-demo          | ⇒ | linux consolen demo                    |
| console-demo-GenICam3 | ⇒ | linux console GenICam demo application |
| filter-driver         | ⇒ | Sphinx linux filter driver             |
| MathParser            | ⇒ | Math parser library                    |
| MathParserSource      | ⇒ | Math parser library source code        |
| sphinxGEVviewer       | ⇒ | linux QT sphinxGEVviewer application   |
| sphinxlib             | ⇒ | Sphinx linux library                   |
| sphinxlib -source     | ⇒ | Sphinx linux library source code       |
| x11-demo              | ⇒ | X-Windows demo                         |

### 3.3.2.2. Install required libraries (as root or using sudo)

Red Hat Enterprise Linux, CentOS, Fedora  
yum install SDL SDL-devel qt4 qt4-devel libxml2 libxml2-devel

Red Hat Enterprise Linux, CentOS, Fedora  
yum install SDL SDL-devel qt4 qt4-devel libxml2 libxml2-devel libxslt libxslt-devel

#### Debian

```
aptitude install libsdl1.2debian libsdl1.2-dev libqt4-core qt4-dev libxml2 libxml2-dev  
libxslt1.1 libxslt1-dev g++
```

#### Ubuntu 12.04

```
apt-get install libsdl1.2debian libsdl1.2-dev libqt4-core libqt4-dev libxml2 libxml2-dev  
libxslt1.1 libxslt1-dev g++
```

#### Ubuntu

```
aptitude install libsdl1.2 libsdl1.2-dev libqt4 libqt4-dev libxml2 libxml2-dev libxslt1.1  
libxslt1.1-dev g++
```

or

```
aptitude install libsdl1.2debian libsdl1.2-dev libqt4-core qt4-dev libxml2 libxml2-dev  
libxslt1.1 libxslt1-dev g++
```

#### openSUSE

```
zypper install SDL SDL-devel qt4 qt4-devel libxml2 libxml2-devel libxslt libxslt-devel g++  
or with yast
```

### **3.3.2.3. Build the MathParser linux library**

Only possible if the MathParser source code (MathParserSource directory) are available.

Command: make or make DEBUG=1 for debug version

Notes: It is necessary to have libxml2, libxslt and zlib development libraries. Also the gnu c++ compiler.

### **3.3.2.4. Build the Sphinx linux library**

Only possible if the SphinxLib source code (sphinxlib-source directory) are available.

Command: make or make DEBUG=1 for debug version

Notes: It is necessary to have libxml2, libxslt and zlib development libraries. Also the gnu c++ compiler.

### **3.3.2.5. Build the linux x-windows demo application**

Command: make

./x11demo or ./x11demo64

### **3.3.2.6. Build the linux console demo application**

Command: make

./demo or ./demo64

---

### **3.3.2.7. Build the linux console GenICam demo application**

Command: `cd Linux64_x64 or Linux32_i86`  
`make`  
`source ./SetGenICamRoot.sh`  
`./demo`

### **3.3.2.8. Build the linux QT sphinxGEVviewer application**

To create a makefile:

32-bit openSUSE, Ubuntu, Debian

Command: `/usr/bin/qmake -unix -o Makefile sphinxGEVviewer.pro`

32-bit Red Hat Enterprise Linux, CentOS, Fedora

Command: `/usr/lib/qt4/bin/qmake -unix -o Makefile sphinxGEVviewer.pro`

64-bit Red Hat Enterprise Linux, CentOS, Fedora, openSUSE

Command: `/usr/lib64/qt4/bin/qmake -unix -o Makefile sphinxGEVviewer.pro`

Command: `make`

Notes: It is necessary to have SDL and QT development libraries. Also the gnu c++ compiler.

### **3.3.2.9. Build the Sphinx linux filter driver**

a) Build the module

Command: `make`

Notes: It is necessary to have the `/usr/src/linux` symbolic link set to point to sources of your current kernel.

b) Install the module

Command: `make install` (must be executed as root)

Notes: It installs the module into current kernel's module directory (`/lib/modules/<kernel_version>/extra`) and runs `depmod` to regenerate dependencies.

Create device nodes `s2igevfilter0.. s2igevfilter9` in `/lib/udev/devices`.

Insert alias `char-major-120 cancamage` to `modprobe.conf` file.

These two steps load the module and create the device node during boot process.

After this steps it is possible to use `s2igevfilter` module.

c) Uninstall the module

Command: `make uninstall` (must be executed as root)

Notes: This uninstalls the module from current kernel's module directory (in my system it is `/lib/modules/<kernel_version>/extra`) and runs `depmod` to regenerate dependencies.

Delete device nodes `s2igevfilter0.. s2igevfilter9` in `/lib/udev/devices`.

## 4. Revision Changes

### 4.1. V2.1.3 to V2.1.4

#### **GEVSetTraversingFirewallsInterval**

---

##### **New**

```
WORD GEVSetTraversingFirewallsInterval(BYTE cam_nr, DWORD interval);
```

#### **GEVGetFeaturePort**

---

##### **New**

```
WORD GEVGetFeaturePort(BYTE cam_nr, char *feature_name, char  
*port_name, int port_name_length);
```

#### **GEVSetSchemaPath**

---

##### **New**

```
WORD GEVGetFeaturePort(BYTE cam_nr, char *schema_path);
```

### 4.2. V2.1.2 to V2.1.3

#### **GEVGetDetailedLog**

---

Supported register read/write log output.

#### **GEVSetDetailedLog**

---

Supported register read/write log output.

#### **GEVTestPacketResend**

---

##### **Old**

```
WORD GEVTestPacketResend(BYTE cam_nr, BYTE on_off, WORD packet_number);
```

##### **New**

```
WORD GEVTestPacketResend(BYTE cam_nr, BYTE on_off, WORD packet_number,  
WORD count);
```

#### **IMAGE\_HEADER**

---

Added PayloadType, ChunkDataPayloadLength and ChunkLayoutId into struct. Used by GEVGetImage function.

### 4.3. V2.1.1 to V2.1.2

No new functions or changes in structures were made.

### 4.4. V2.1.0 to V2.1.1

#### **FEATURE\_PARAMETER**

---

Added PollingTime into struct. Used by GEVGetFeatureParameter function.

#### **GEVTestFindMaxPacketSize**

---

##### **Old**

```
WORD GEVTestFindMaxPacketSize(BYTE cam_nr, WORD *packet_size, WORD  
ps_min, WORD ps_max, WORD ps_inc);
```

**New**

```
WORD WINAPI GEVTestFindMaxPacketSize(BYTE cam_nr, WORD *packet_size,  
WORD ps_min, WORD ps_max, WORD ps_inc);
```

## 4.5. V2.0.9 to V2.1.0

### GEVGetXmlSize

---

**New**

```
WORD WINAPI GEVGetXmlSize(BYTE cam_nr, DWORD *size);
```

### GEVGetXmlFile

---

**New**

```
WORD WINAPI GEVGetXmlFile(BYTE cam_nr, BYTE **xmlfile);
```

### GEVAcquisitionStartEx

---

**New**

```
WORD WINAPI GEVAcquisitionStartEx(BYTE cam_nr, DWORD  
number_images_to_acquire, DWORD image_size);
```

### GEVGetPacketsOutOfOrder

---

**New**

```
WORD WINAPI GEVGetPacketsOutOfOrder(BYTE cam_nr, BYTE  
*packets_out_of_order);
```

### GEVSetPacketsOutOfOrder

---

**New**

```
WORD WINAPI GEVSetPacketsOutOfOrder(BYTE cam_nr, BYTE  
packets_out_of_order);
```

### GEVTestFindMaxPacketSize

---

**Old**

```
WORD GEVTestFindMaxPacketSize(BYTE cam_nr, WORD *packet_size);
```

**New**

```
WORD GEVTestFindMaxPacketSize(BYTE cam_nr, WORD *packet_size, WORD  
ps_min, WORD ps_max, WORD ps_inc);
```

### GEVGetFeatureInvalidator

---

**New**

```
WORD WINAPI GEVGetFeatureInvalidator(BYTE cam_nr, char *feature_name,  
BYTE index, char *invalidator_name, int str_len);
```

### GEVSetReadWriteMemoryCallback

---

**New**

```
WORD WINAPI GEVSetReadWriteMemoryCallback(BYTE cam_nr,  
READ_WRITE_MEM_CALLBACK_FUNC c_func);
```

---

## CANCamSendClink

---

### Old

```
WORD WINAPI CANCamSendClink(BYTE cam_nr, BYTE count, BYTE  
*send_buffer);
```

### New

```
WORD WINAPI CANCamSendClink(BYTE cam_nr, WORD count, BYTE  
*send_buffer);
```

---

## CANCamReceiveClink

---

### Old

```
WORD WINAPI CANCamReceiveClink(BYTE cam_nr, BYTE count_buffer, BYTE  
*recv_buffer, BYTE *recv_count);
```

### New

```
WORD WINAPI CANCamReceiveClink(BYTE cam_nr, WORD count_buffer, BYTE  
*recv_buffer, WORD *recv_count);
```

## 4.6. V2.0.8 to V2.0.9

---

## MESSAGECHANNEL\_CALLBACK\_FUNC

---

### Old

```
typedef BYTE ( WINAPI *MESSAGECHANNEL_CALLBACK_FUNC ) (BYTE cam_nr, int  
event_id, int data_length, BYTE *data);
```

### New

```
typedef BYTE ( WINAPI *MESSAGECHANNEL_CALLBACK_FUNC ) (BYTE cam_nr,  
MESSAGECHANNEL_PARAMETER mcparam);
```

---

## GEVGetPixelPtr

---

### Old

```
WORD GEVGetPixelPtr(BYTE cam_nr, DWORD offset, DWORD *ptr_adr);
```

### New

```
WORD GEVGetPixelPtr(BYTE cam_nr, DWORD offset, UINT64 *ptr_adr);
```

---

## GEVGetImageBuffer

---

### New

```
WORD WINAPI GEVGetImageBuffer(BYTE cam_nr, IMAGE_HEADER *image_header,  
BYTE *image_buffer);
```

---

## GEVSetRingBuffer

---

### New

```
WORD WINAPI GEVSetRingBuffer(BYTE cam_nr, WORD index, void *buffer);
```

---

## GEVQueueRingBuffer

---

### New

```
WORD WINAPI (BYTE cam_nr, BYTE image_buffer_index);
```

---

## GEVReleaseRingBuffer

---

### New

```
WORD WINAPI GEVReleaseRingBuffer(BYTE cam_nr);
```

---

## GEVGetImageRingBuffer

---

### New

```
WORD WINAPI GEVGetImageRingBuffer(BYTE cam_nr, IMAGE_HEADER  
*image_header, BYTE *image_buffer_index);
```

## 4.7. V2.0.7 to V2.0.8

---

## GEVSetSecureTransfer

---

### New

```
WORD WINAPI GEVSetSecureTransfer(BYTE cam_nr, BYTE enable,  
SECURE_TRANSFER_CALLBACK_FUNC c_func);
```

---

## GEVGetSecureTransfer

---

### New

```
WORD WINAPI GEVGetSecureTransfer(BYTE cam_nr, BYTE *enable);
```

## 4.8. V2.0.5 to V2.0.6

---

## GEVSetBufferCount

---

### New

```
WORD WINAPI GEVSetBufferCount(BYTE cam_nr, WORD count);
```

---

## GEVGetBufferCount

---

### New

```
WORD WINAPI GEVGetBufferCount(BYTE cam_nr, WORD *count);
```

---

## GEVSetActionCommand

---

### New

```
WORD WINAPI GEVSetActionCommand(BYTE cam_nr, DWORD device_key, DWORD  
group_key, DWORD group_mask, DWORD action_time)
```

## 4.9. V2.0.4 to V2.0.5

---

## MESSAGECHANNEL\_CALLBACK\_FUNC

---

### Old

```
typedef BYTE ( WINAPI *MESSAGECHANNEL_CALLBACK_FUNC )(int event_id, int  
data_length, BYTE *data);
```

### New

```
typedef BYTE ( WINAPI *MESSAGECHANNEL_CALLBACK_FUNC )(BYTE cam_nr, int  
event_id, int data_length, BYTE *data);
```

---

## ERROR\_CALLBACK\_FUNC

---

### Old



```
typedef BYTE ( WINAPI *ERROR_CALLBACK_FUNC ) (char *error_str);
```

#### New

```
typedef BYTE ( WINAPI *ERROR_CALLBACK_FUNC ) (BYTE cam_nr, char *error_str);
```

## 4.10. V2.0.2 to V2.0.3

### GEVInit

#### Old

```
WORD WINAPI GEVInit(BYTE cam_nr, CONNECTION *con, ERROR_CALLBACK_FUNC error_func, BYTE save_xml);
```

```
con.AdapterIP = dis.param[camera-1].AdapterIP;
con.AdapterMask = dis.param[camera-1].AdapterMask;
con.IP_CANCam = htonl(dis.param[camera-1].IP);
con.PortCtrl = 2000;
con.PortData = 2001;
strcpy(con.adapter_name,dis.param[camera-1].adapter_name);
```

```
error = GEVInit(camera, &con, NULL, 0);
```

#### New

```
WORD WINAPI GEVInit(BYTE cam_nr, CONNECTION *con, ERROR_CALLBACK_FUNC error_func, BYTE save_xml, BYTE open_mode);
```

```
con.AdapterIP = dis.param[camera-1].AdapterIP;
con.AdapterMask = dis.param[camera-1].AdapterMask;
con.IP_CANCam = dis.param[camera-1].IP;
```

```
con.PortCtrl = 2000; // set static port
con.PortData = 2001; // set static port
or
con.PortCtrl = 0;    // set port automatically
con.PortData = 0;    // set port automatically
```

```
strcpy(con.adapter_name,dis.param[camera-1].adapter_name);
```

```
error = GEVInit(camera, &con, NULL, 0,EXCLUSIVE_ACCESS);
```

| Open Mode        | Value |
|------------------|-------|
| OPEN_ACCESS      | 0     |
| EXCLUSIVE_ACCESS | 1     |
| CONTROL_ACCESS   | 2     |

### GEVOpenStreamChannel

#### Old

```
WORD WINAPI GEVOpenStreamChannel(BYTE cam_nr, DWORD ip, WORD port);
```

```
error = GEVOpenStreamChannel(camera, con.AdapterIP, con.PortData);
```

#### New

```
WORD WINAPI GEVOpenStreamChannel(BYTE cam_nr, DWORD ip, WORD port,
DWORD multicast);
error = GEVOpenStreamChannel(camera, con.AdapterIP, con.PortData,0
);
```

### GEVAcquisitionStart

#### Old

```
WORD WINAPI GEVAcquisitionStart(BYTE cam_nr);
error = GEVAcquisitionStart(camera);
```

#### New

```
WORD WINAPI GEVAcquisitionStart(BYTE cam_nr, DWORD
number_images_to_acquire);
```

```
acquisition_counter = 0; // start acquisition in the continuous mode
error = GEVAcquisitionStart(camera,acquisition_counter);
```

| AcquisitionMode | number_images_to_acquire                                  |
|-----------------|-----------------------------------------------------------|
| Continuous      | 0 (unlimited)                                             |
| SingleFrame     | 1                                                         |
| MultiFrame      | 1..n<br>Get the value from the AcquisitionFrameCount node |

## 4.11. V1.x.x to V2.x.x

| V1.x.x              | V2.x.x                               |
|---------------------|--------------------------------------|
| discovery_cancam    | GEVDiscovery                         |
| init_cancam         | GEVInit                              |
| close_cancam        | GEVClose                             |
| init_filter_driver  | GEVInitFilterDriver                  |
| close_filter_driver | GEVCloseFilterDriver                 |
| send_val            | GEVWriteRegister                     |
| receive             | GEVReadRegister                      |
| get_pixel_ptr       | GEVGetPixelPtr                       |
| debug_info          | GEVGetDetailedLog, GEVSetDetailedLog |
| set_memory_size     | GEVSetMemorySize                     |
| get_memory_size     | GEVGetMemorySize                     |
| set_lut             | CANCamSetLut                         |
| get_lut             | CANCamGetLut                         |

|                              |                              |
|------------------------------|------------------------------|
| set_message_channel_callback | GEVSetMessageChannelCallback |
| get_local_error              | not available                |
| receive_memory               | GEVWriteMemory               |
| send_memory                  | GEVReadMemory                |
| set_net_config               | GEVSetNetConfig              |
| get_net_config               | GEVGetNetConfig              |
| force_ip                     | GEVForceIp                   |
| set_read_write_parameter     | GEVSetReadWriteParameter     |
| get_read_write_parameter     | GEVGetReadWriteParameter     |
| set_heartbeat_rate           | GEVSetHeartbeatRate          |
| get_heartbeat_rate           | GEVGetHeartbeatRate          |
| test_packet                  | GEVTestPacket                |
| get_filter_driver_version    | GEVGetFilterDriverVersion    |
| set_channel_parameter        | GEVSetChannelParameter       |
| get_channel_parameter        | GEVGetChannelParameter       |
| ---                          | GEVOpenStreamChannel         |
| ---                          | GEVCloseStreamChannel        |
| ---                          | GEVInitXml                   |
| get_root_count               | GEVGetFeatureList            |
| get_root_name                | GEVGetFeatureList            |
| get_feature_count            | GEVGetFeatureList            |
| get_feature_name             | GEVGetFeatureList            |
| get_sub_feature_name         | GEVGetFeatureList            |
| get_sub_feature_count        | GEVGetFeatureList            |
| get_subsub_feature_count     | GEVGetFeatureList            |
| get_subsub_feature_name      | GEVGetFeatureList            |
| get_feature_parameter        | GEVGetFeatureParameter       |
| get_feature_integer          | GEVGetFeatureInteger         |
| set_feature_integer          | GEVSetFeatureInteger         |
| get_feature_string           | GEVGetFeatureString          |
| set_feature_string           | GEVSetFeatureString          |
| get_feature_boolean          | GEVGetFeatureBoolean         |
| set_feature_boolean          | GEVSetFeatureBoolean         |
| get_feature_command          | GEVGetFeatureCommand         |
| set_feature_command          | GEVSetFeatureCommand         |
| set_feature_enumeration      | GEVSetFeatureEnumeration     |
| get_feature_enumeration      | GEVGetFeatureEnumeration     |
| get_feature_enumeration_name | GEVGetFeatureEnumerationName |
| get_feature_float            | GEVGetFeatureFloat           |
| set_feature_float            | GEVSetFeatureFloat           |
| get_feature_display_name     | GEVGetFeatureDisplayName     |
| get_feature_tooltip          | GEVGetFeatureTooltip         |
| set_xml_file                 | GEVSetXmlFile                |
| get_feature_register         | GEVGetFeatureRegister        |
| set_feature_register         | GEVSetFeatureRegister        |
| get_feature_unit             | GEVGetFeatureUnit            |
| get_feature_enable_status    | GEVGetFeatureEnableStatus    |
| start_grab                   | GEVAcquisitionStart          |
| stop_grab                    | GEVAcquisitionStop           |
| grab_image                   | GEVGetImage                  |
| get_image_header             | GEVGetImage                  |
| set_packet_resend            | GEVSetPacketResend           |
| get_packet_resend            | GEVGetPacketResend           |

|                                      |                             |
|--------------------------------------|-----------------------------|
| get_image_fps                        | GEVGetImageFPS              |
| ---                                  | GEVPacketResend             |
| ---                                  | GEVGetImageFPS              |
| ---                                  | GEVSetBufferCount           |
| ---                                  | GEVGetBufferCount           |
| ---                                  | GEVTestPacketResend         |
| ---                                  | GEVTestFindMaxPacketSize    |
| // these functions should not use it |                             |
| get_pixel_format                     | CANCamGetPixelFormat        |
| get_max_video_window                 | CANCamGetMaxVideoWindow     |
| get_video_window                     | CANCamGetVideoWindow        |
| set_video_window                     | CANCamSetVideoWindow        |
| get_gain                             | CANCamGetGain               |
| set_gain                             | CANCamSetGain               |
| get_gain_min_max                     | CANCamGetGainMinMax         |
| set_gain_auto                        | CANCamSetGainAuto           |
| get_exposure_time                    | CANCamGetExposureTime       |
| set_exposure_time                    | CANCamSetExposureTime       |
| get_exposure_time_min_max            | CANCamGetExposureTimeMinMax |
| set_exposure_time_auto               | CANCamSetExposureTimeAuto   |